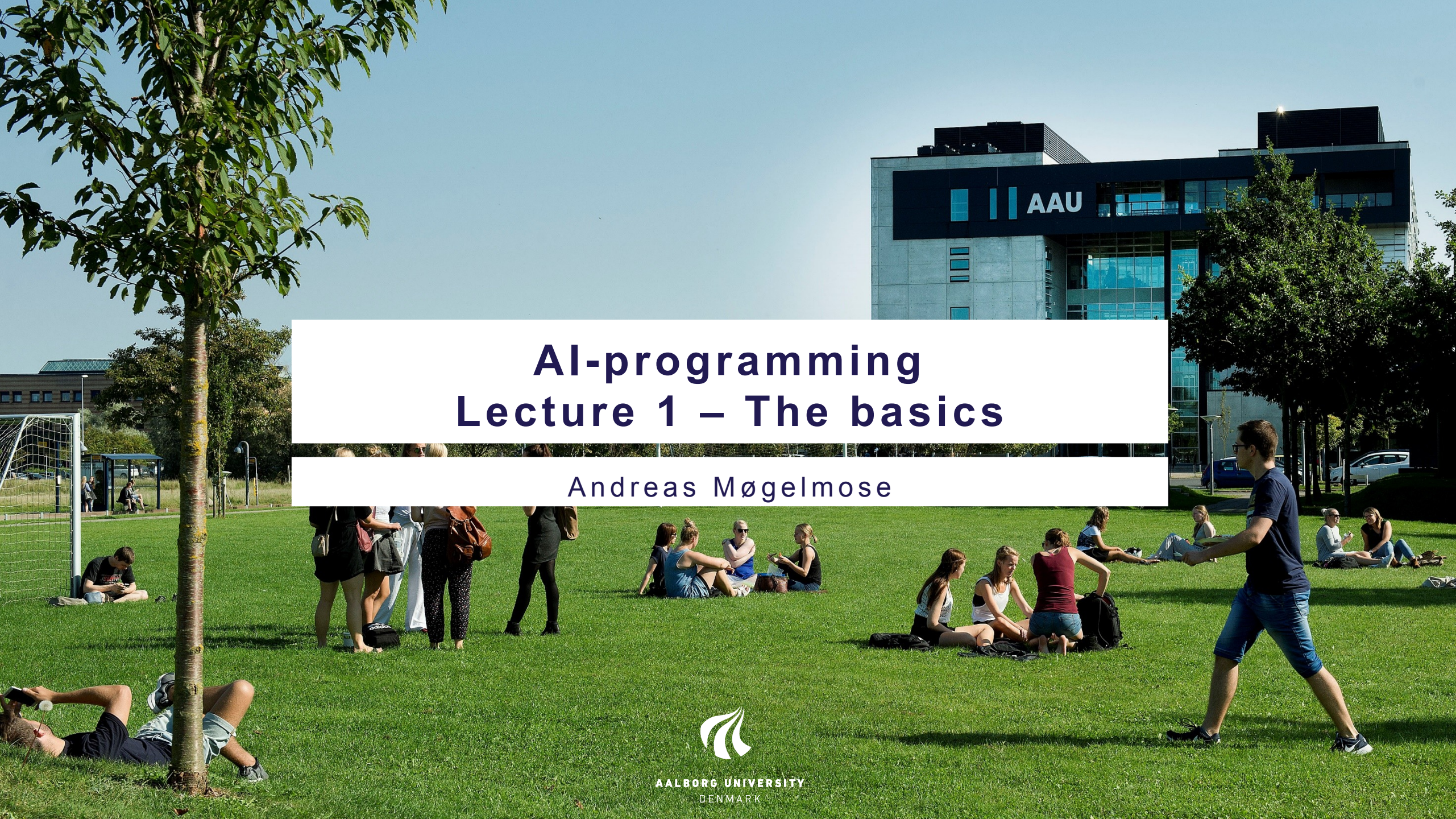


AI-programming
Lecture 1 – The basics

Andreas Møgelmose

AALBORG UNIVERSITY
DENMARK



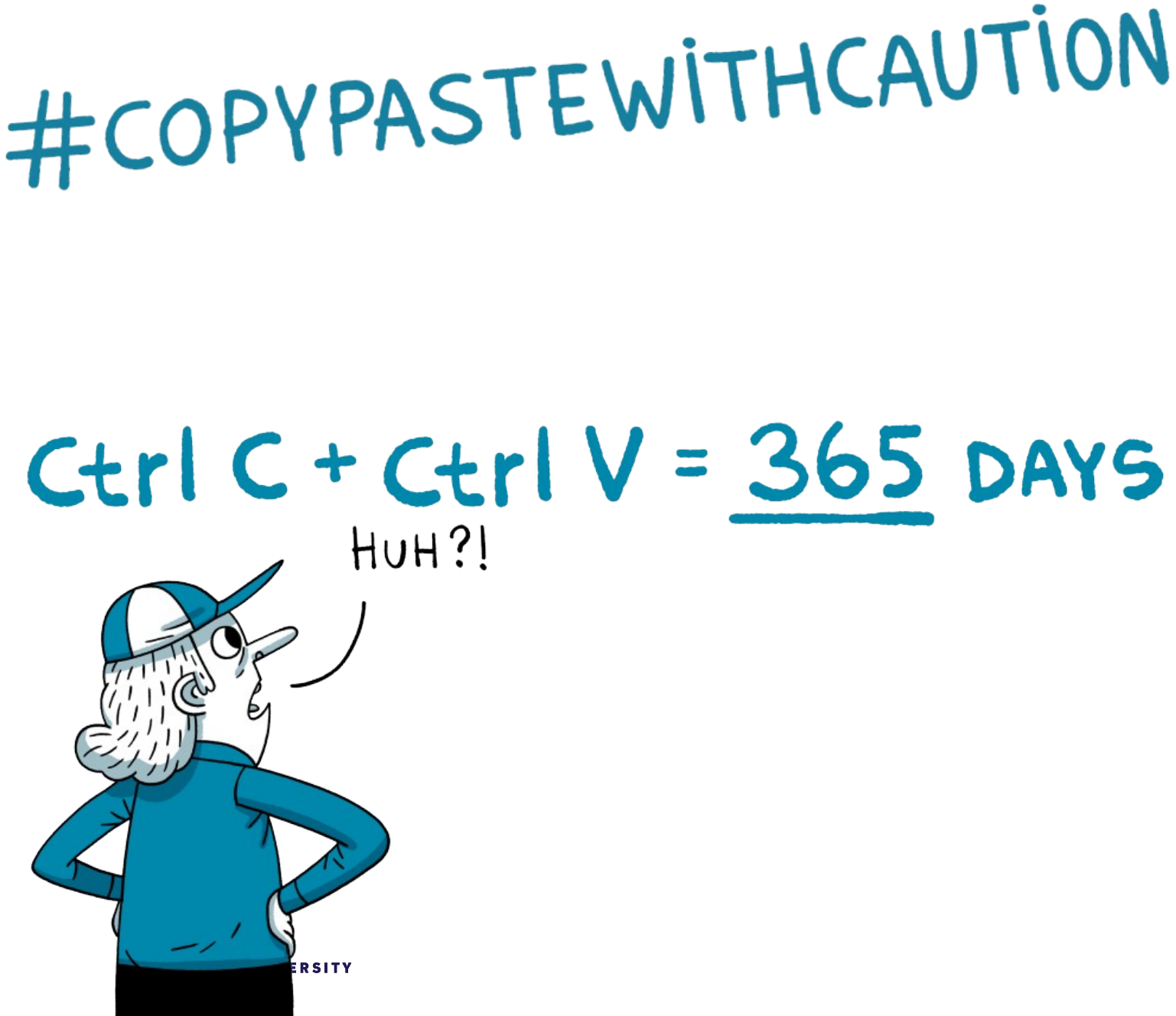
Information fra dit studie

CREATE semesterstart, september 2023

AALBORG UNIVERSITET

Plagiat

- Plagiat er forbudt og har store konsekvenser.
- Brug af egne tidligere aflevering uden henvisning kaldes selvplagiat og er også forbudt.
- Alle afleveringer tjekkes for plagiat af AAUs plagiatkontrol.
- AI-genereret tekst kan også identificeres i plagiatkontrol.
- Bortvisning i et halvt eller helt år er den typiske sanktion.



Plagiat

- Studerende har selv ansvar for at holde sig opdaterede omkring regler for plagiat og eksamenssnyd.

- Læs mere her: https://www.studerende.aau.dk/studie-trivselsvejledning/regler/plagiat/



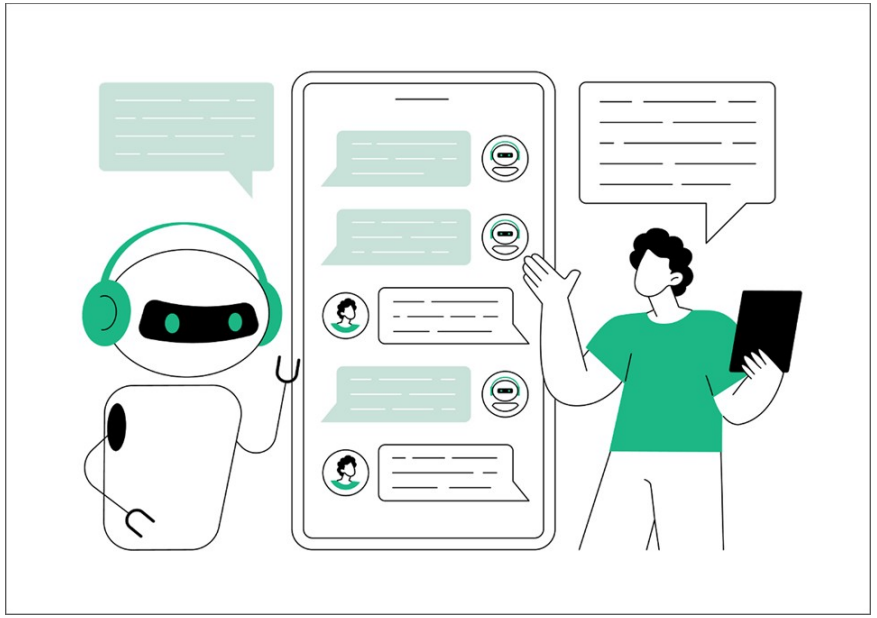
Generativ AI – AAU Fælles Regler

- Sæt dig ind i reglerne: https://www.studerende.aau.dk/praktisk/it/generativ-ai-pa-aau

- Du må anvende AI til:
  - Idegenerering
  - Informationssøgning (husk korrekt kildeangivelse)
  - Generering af billeder, video og lyd med korrekt kildeangivelse (vær opmærksom på licensforhold)

- Du må **ikke** anvende AI til:
  - generering af tekst
  - omarbejde tekst
  - generering af source code

- **Brug af AI genereret tekst i en projektrapport betragtes som plagiat!**



AALBORG UNIVERSITET

Regler fra studienævnet

- Medieteknologi:
  - Generativ AI er godkendt som hjælpemiddel til alle projektrapporter, men studerende skal tydelig markere at det blevet brugt og reflektere over brugen.
  - Til kurser er det kursuskoordinator de må beslutter om generativ AI er godkendt (skal angives under ”godkendte hjælpemidler”)

Agenda

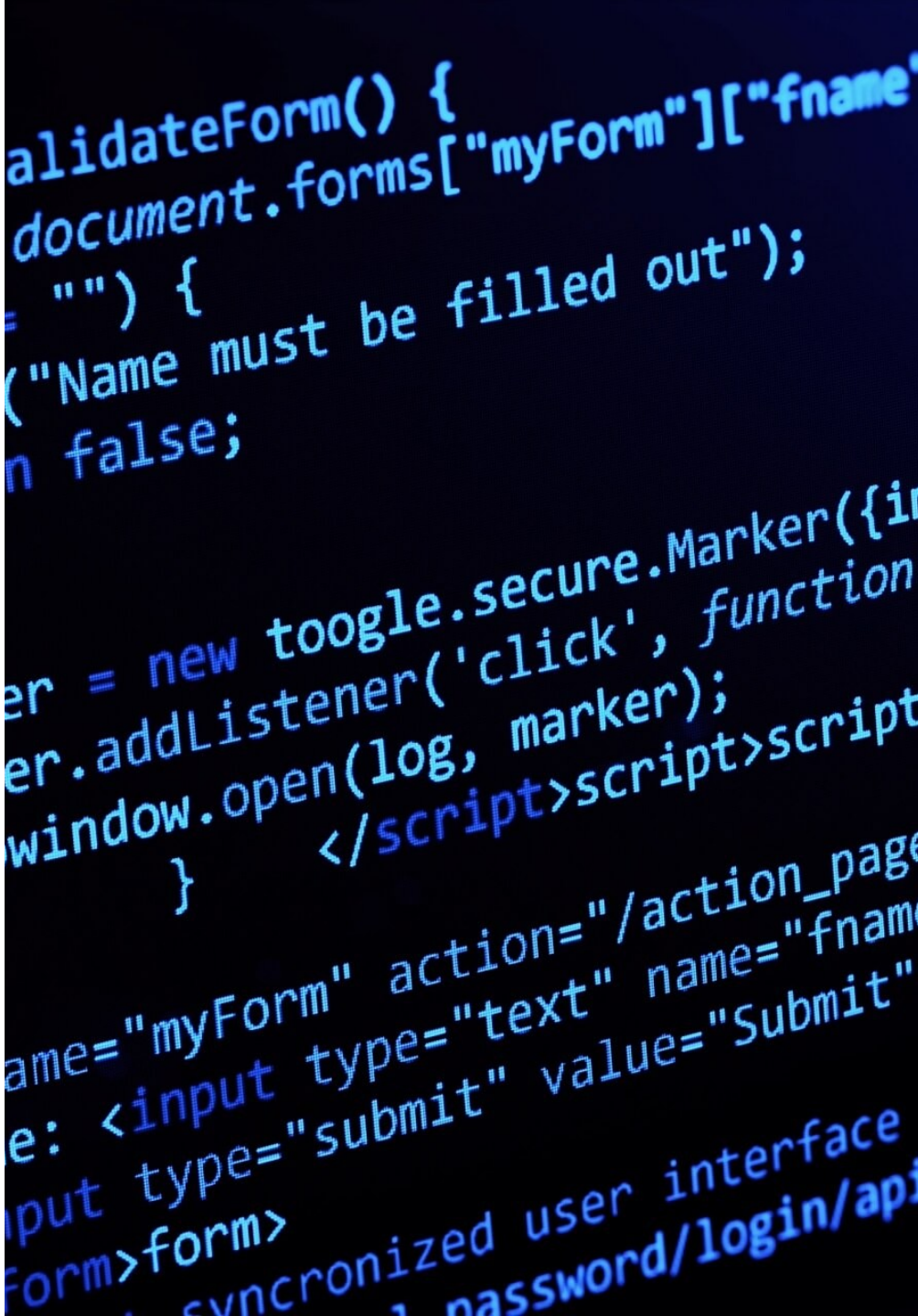
- Course structure
- What is programming even?
- What are block diagrams?
- Getting started with coding:
  - Calling functions
  - Sharing code with Git
  - Looking at documentation



What will we learn?

- **Thinking in programming**: Flowcharts and comments
- Doing programming: Python
- Applying programming: AI algorithms

- We’re quite ambitious on your behalf!



AALBORG UNIVERSITY
DENMARK

WARNING: Many students fail this course!

- 2023: 18%
- 2024: 19%

- Attend lectures
- Spend time on exercises
- Practice, practice, practice (without ChatGPT)
- ASK FOR HELP



AALBORG UNIVERSITY
DENMARK

Books and resources



Structure of lectures

- Before lecture:
  - Read the assigned material/follow tutorials
- During scheduled lectures (lecture hall and group rooms):
  - Lecture: 30-60 minutes in auditorium
  - Solve exercises
  - Work on assignments
  - Optional: Come back at 11 to see me live-code an exercise from last lecture
- Homework:
  - Finish exercises and quizzes
  - Hand in assignments by deadlines

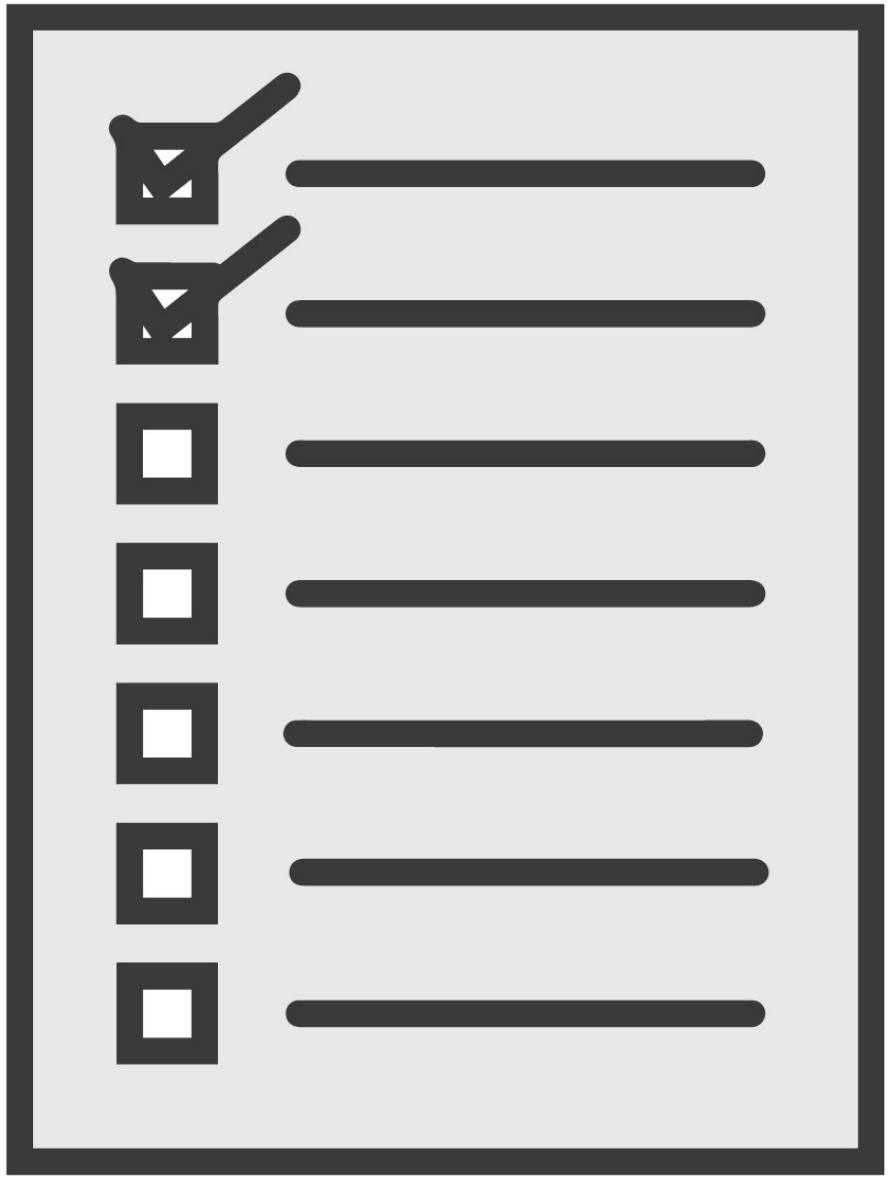


Exercises

- Exercise session after each lecture
- Exercises are done on your own computer in the group rooms
- Two types of exercises
  - Small exercises directly related to the topic of the day
  - 2 mini-projects
- Final mini-project is the foundation for the exam

- Exercises are not checked by us. They are only provided to help you learn the material. But please do them anyway
  - Mini-projects, however, are mandatory.

- You are encouraged to talk and **help each other** in the groups



Mini-projects

- 2 mandatory mini-projects
- What if I miss a deadline because of [insert your excuse]??
  - Open for late hand-ins exactly one week after original deadline
  - If you need an extension for both projects, the penalty is an extra mini-project, i.e., 3 mini-projects need to be approved!
  - If you miss the extended deadline after a 1 week extension, you fail the exam.



AALBORG UNIVERSITY
DENMARK

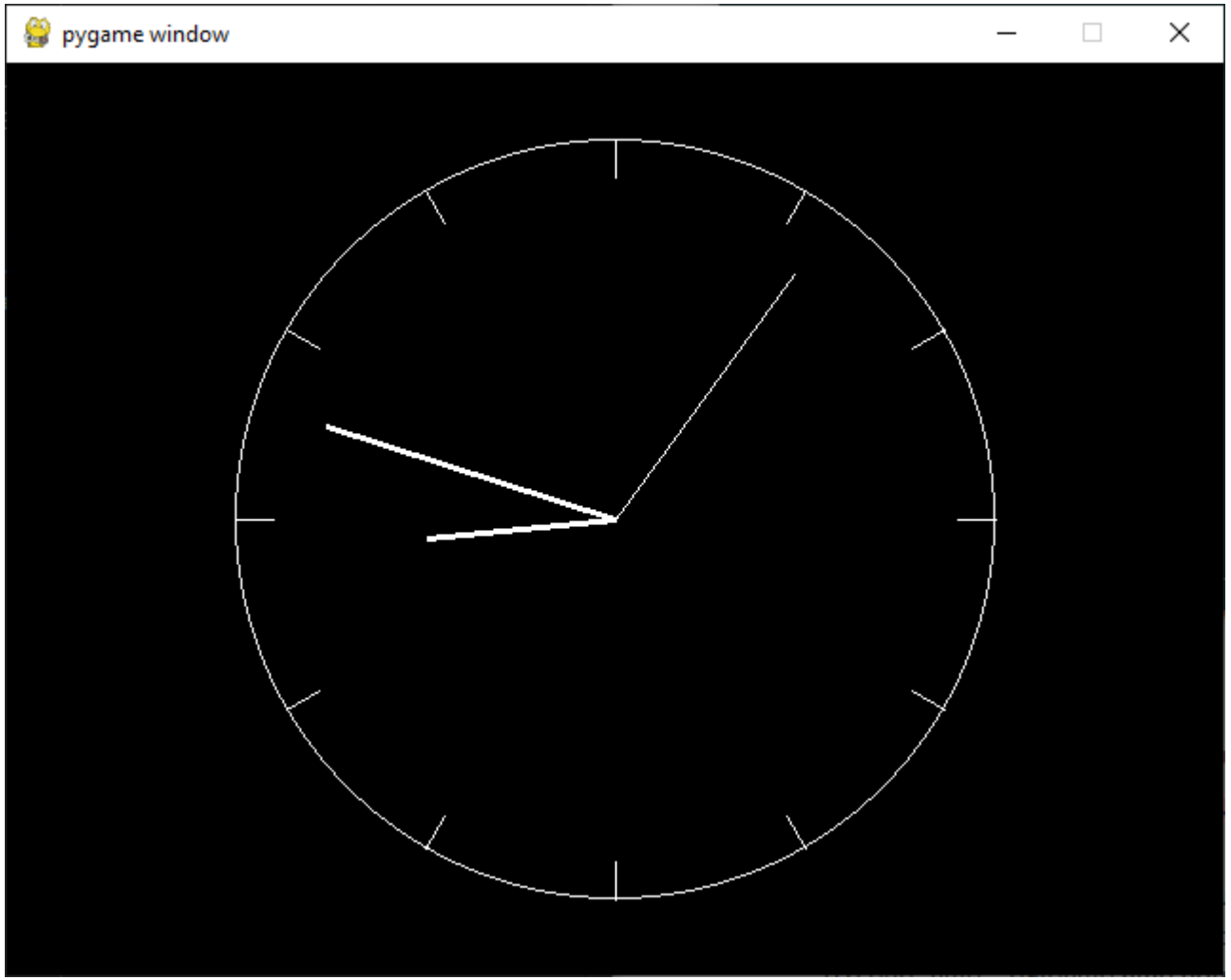
Mini-projects

- They must be done individually!
- The hand-ins consist of your own code
  - You cannot copy/paste from other students, the internet, ChatGPT.
  - You may be inspired by tutorials, but not follow them directly



Mini-project 1

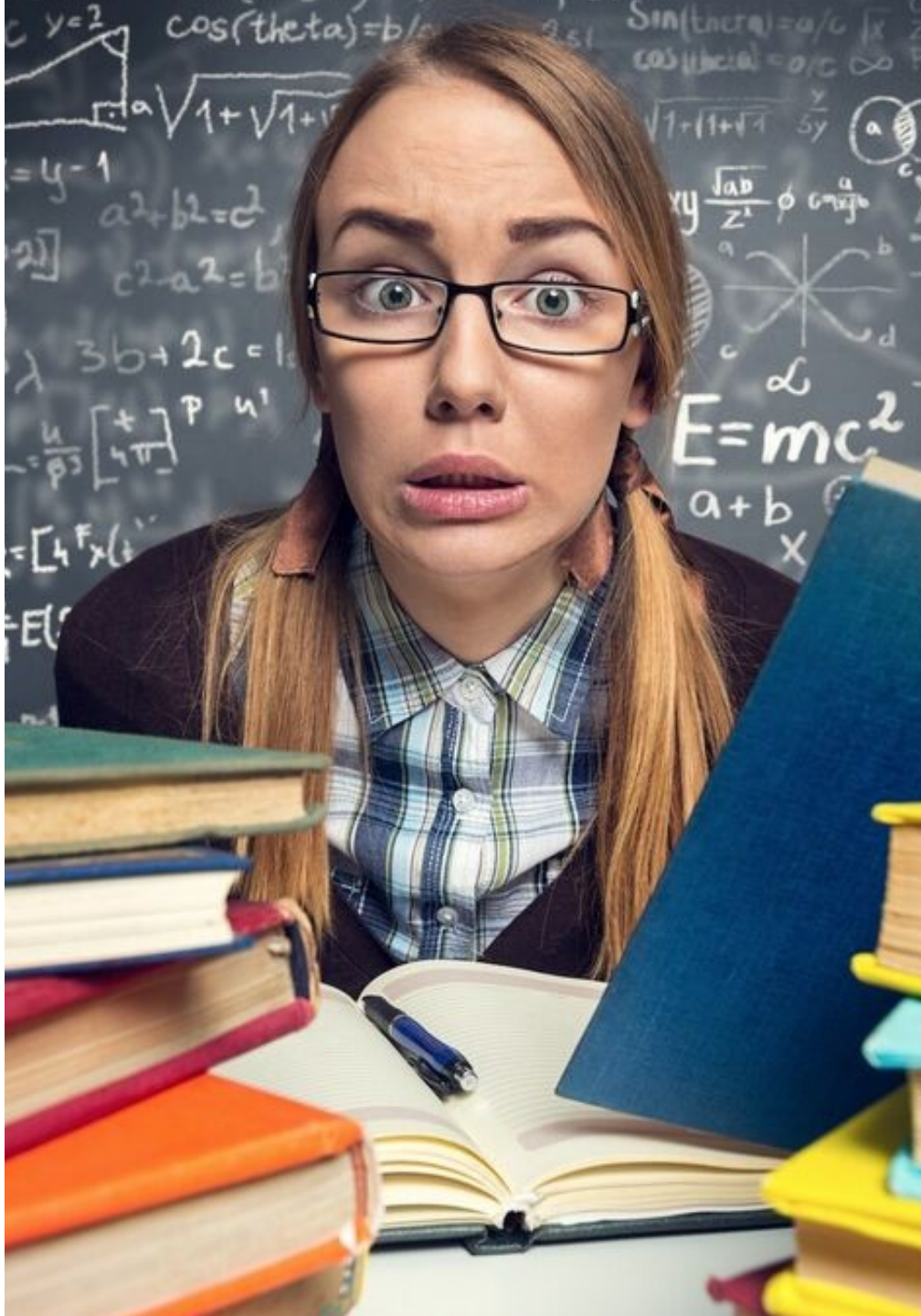
- Make an analog clock
- The clock must have 12 hour-markings, and moving hour-, minute-, and second-arms.
- The clock must show the current time.
- The rest is up to you – have fun



The bare minimum :-)

Final mini-project

- Robot navigation with A*
- Individual hand-in
- Code + assets (if necessary). No report.
- Oral exam (20 minutes)



Agenda

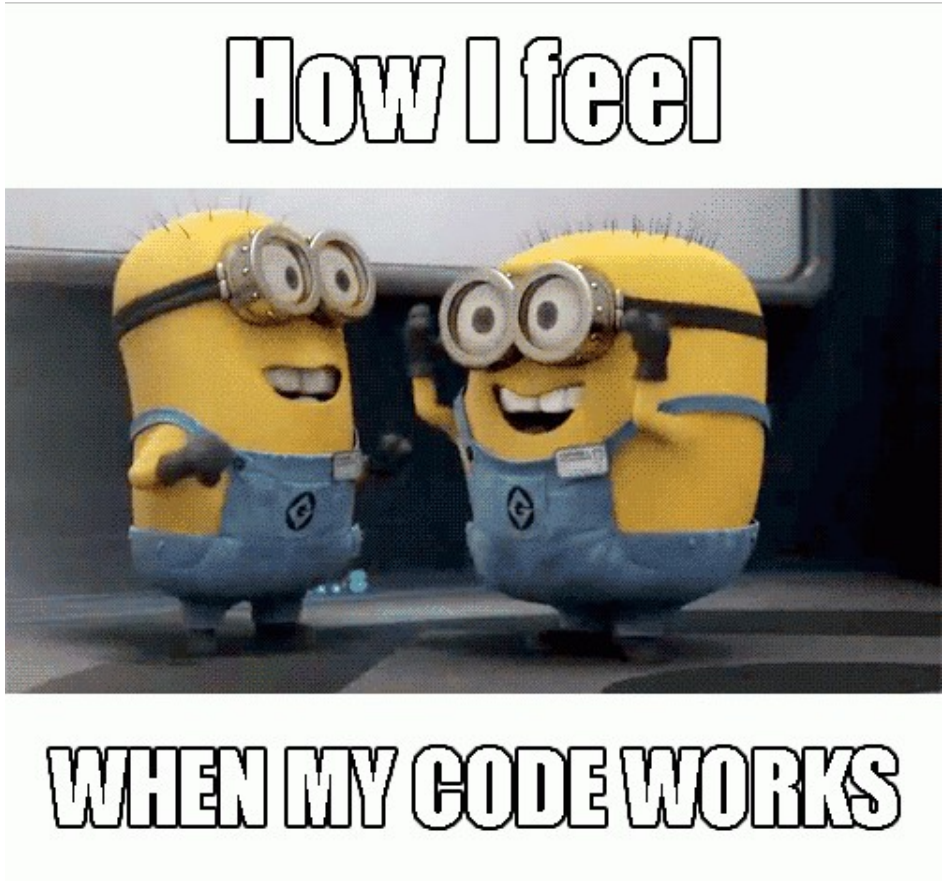
- Course structure
- **What is programming even?**
- What are block diagrams?
- Getting started with coding: Calling functions



Why program?

- Great fun
- Immensely creative – you start with literally nothing and create stuff out of thin air
- Challenging

- A great tool, even if you're not a professional programmer (but you guys will be…!)



Why program?

- To build AI-systems, you need to be able to code.
- You will be programming for every semester project and most courses!
- In other words: Programming is the foundation of your entire education – and perhaps career!
- … but mostly: It's great fun!



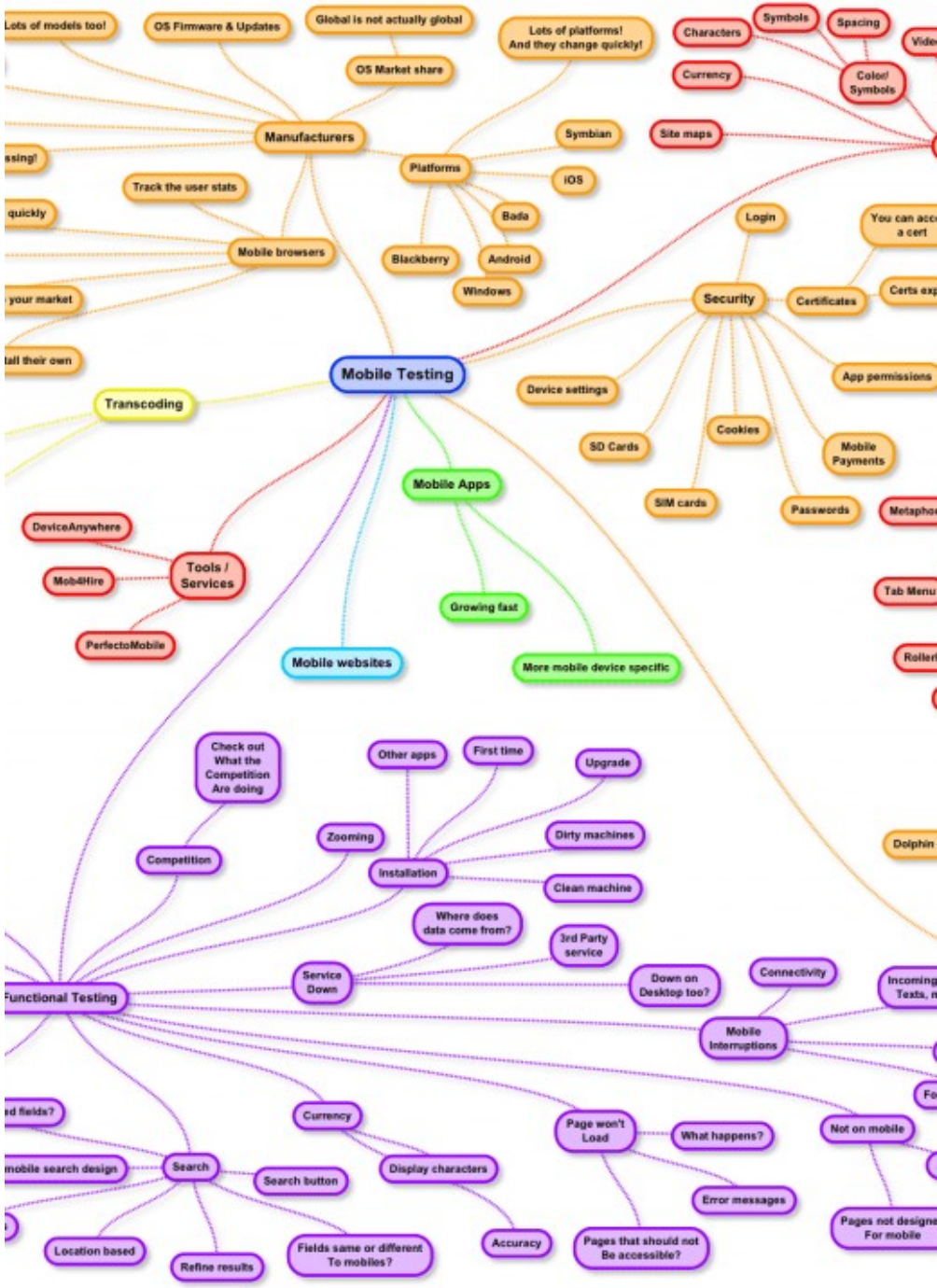
Is programming difficult?



(Why) is programming difficult?

- Programming is a mix of many things:
  - Syntax (the language)
  - Breaking down and describing problems
  - Tools and tools and tools
    - Compilers
    - Editors
    - Version control
  - Libraries
  - Terminology

- Do not try to understand it all at once!

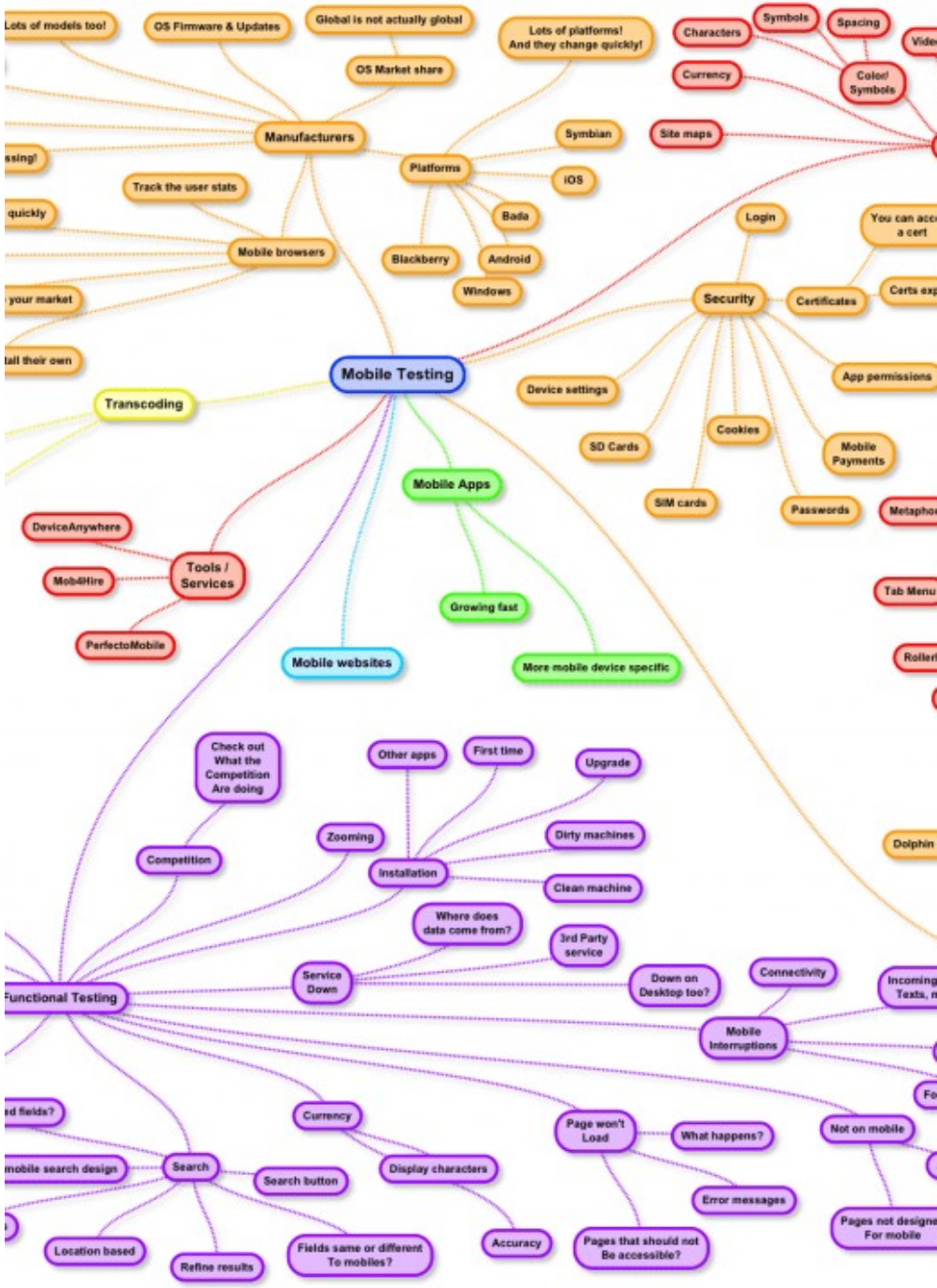


AALBORG UNIVERSITY
DENMARK

(Why) is programming difficult?

- Programming is a mix of many things:
  - Syntax (the language)
  - **Breaking down and describing problems**
  - Tools and tools and tools
    - Compilers
    - Editors
    - Version control
  - Libraries
  - Terminology

- Do not try to understand it all at once!



AALBORG UNIVERSITY
DENMARK

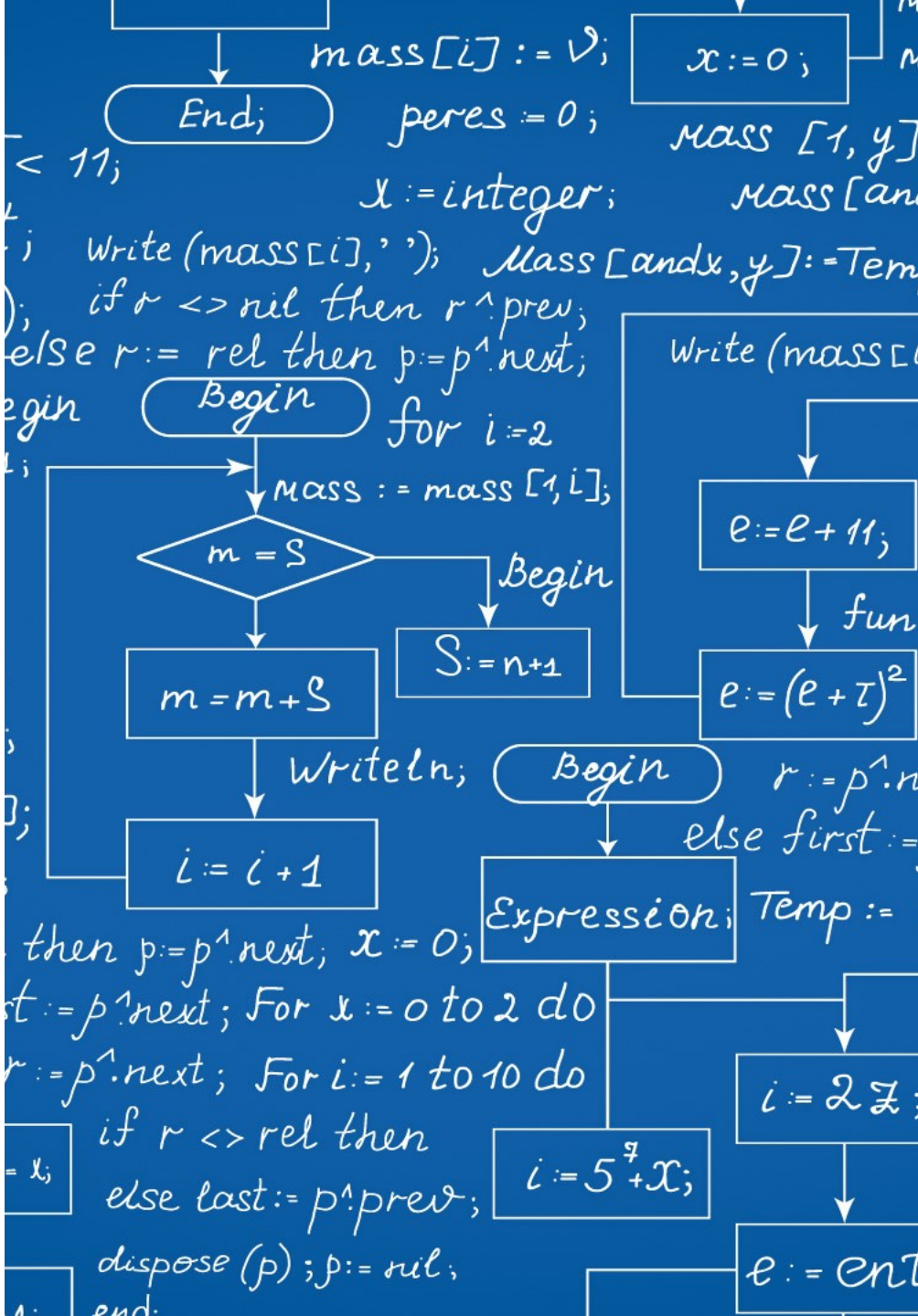
What is programming?

- Programming = the activity that produces computer programs
- Programming = giving a series of instructions to a computer (code)
  - Instructions are carried out in order, unless you tell the computer to do otherwise
  - All instructions are basically about reading, writing, or manipulating information
  - Maybe some of you have used Maple or similar software. This is programming. Excel formulas are programming.



Algorithms

- What does it mean that instructions are carried out ‘in order’?
  - It means that the computer treats the instructions much in the same way as you treat words and pages in a book
    - You read words in a specific order, e.g. left-to-right, top-to-bottom
    - A computer typically executes all instructions in one line, before proceeding to the next line
  - A series of instructions to accomplish a specific task is called an *algorithm*
  - *Programming makes algorithms understandable to computers*



AALBORG UNIVERSITY
DENMARK

Algorithms

- Which leads us to …

The first step in learning to program a computer, is to be able to describe an algorithm!

Do you know any algorithms already?



Algorithms

- Cooking

Baked Beef Stew

Directions

1. Preheat oven to 375 degrees F (190 degrees C).
2. Lightly grease a 9x13 inch baking dish.
3. In a large skillet over medium heat, brown the stew meat; drain and set aside.
4. In a mixing bowl, combine the tomatoes, water, tapioca, beef bouillon granules, sugar, salt and pepper.
5. Stir in the beef, carrots, celery, potatoes, onion, and bread cubes. Pour into the prepared baking dish.
6. Cover and bake for 2 hours, or until meat and vegetables are tender.

From allrecipes.com



Algorithms

- Knitting

Alexa Bernadette Dress

STRIPE PATTERN
Working in St st, * work 6 rows A, then 2 rows B. Rep from * for Stripe Pat.

BACK
With larger needles, using A, cast on 43(46-49-52) sts. Knit 2 rows. Work even in Stripe Pat until piece measures 9(9 1/2-10-10 1/2)", end on WS.
Shape Bodice: Knit the next row, dec 15 sts evenly spaced across – 28(31-34-37) sts. Work even in Stripe Pat until bodice measures 2" above dec row, end on WS. Mark beg and end of last row for beg of armholes.
Shape Armholes: Bind off 2 sts at beg of the next 2 rows. Dec 1 st at each armhole edge every RS row once. Work even on 22(25-28-31) sts until armholes measure 2 1/2(3-3 1/2-4)", end on WS.
...

From www.berroco.com



Algorithms

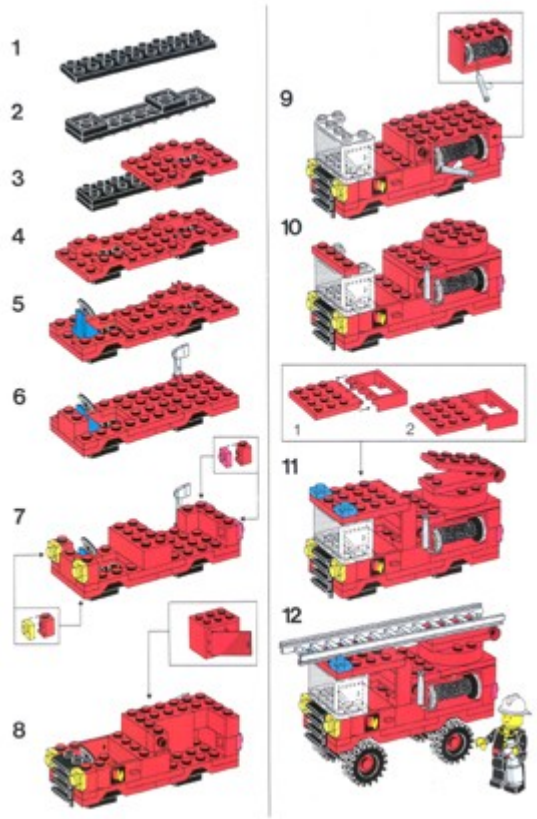
- Blackjack rules (simplified)

- Deal two cards to every player
- Each player makes a decision to hit, stand, double down, split, or surrender
- When all players have decided to stand
  - If any player or the dealer exceeds 21, he loses
  - If a player or the dealer gets a total of 21, he wins
  - If all players choose to stand, the one closest to 21 wins



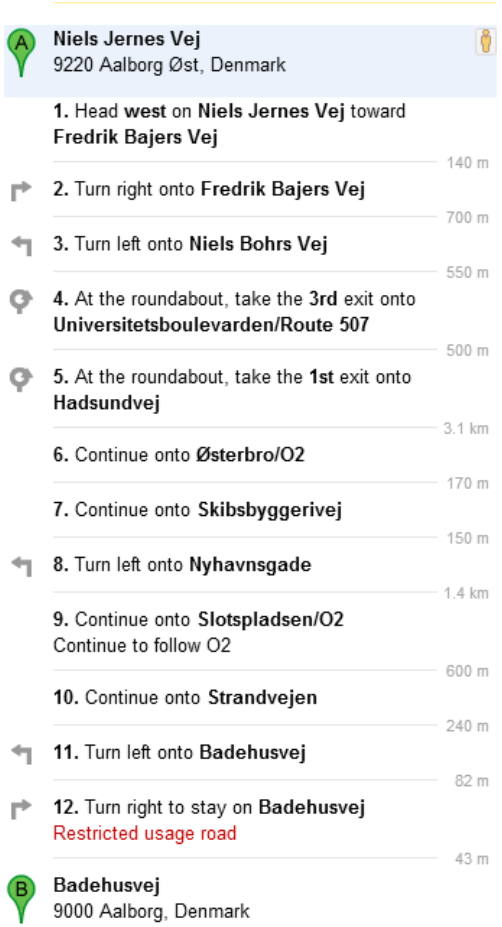
Algorithms

- Lego assembly guides



Algorithms

- Driving directions



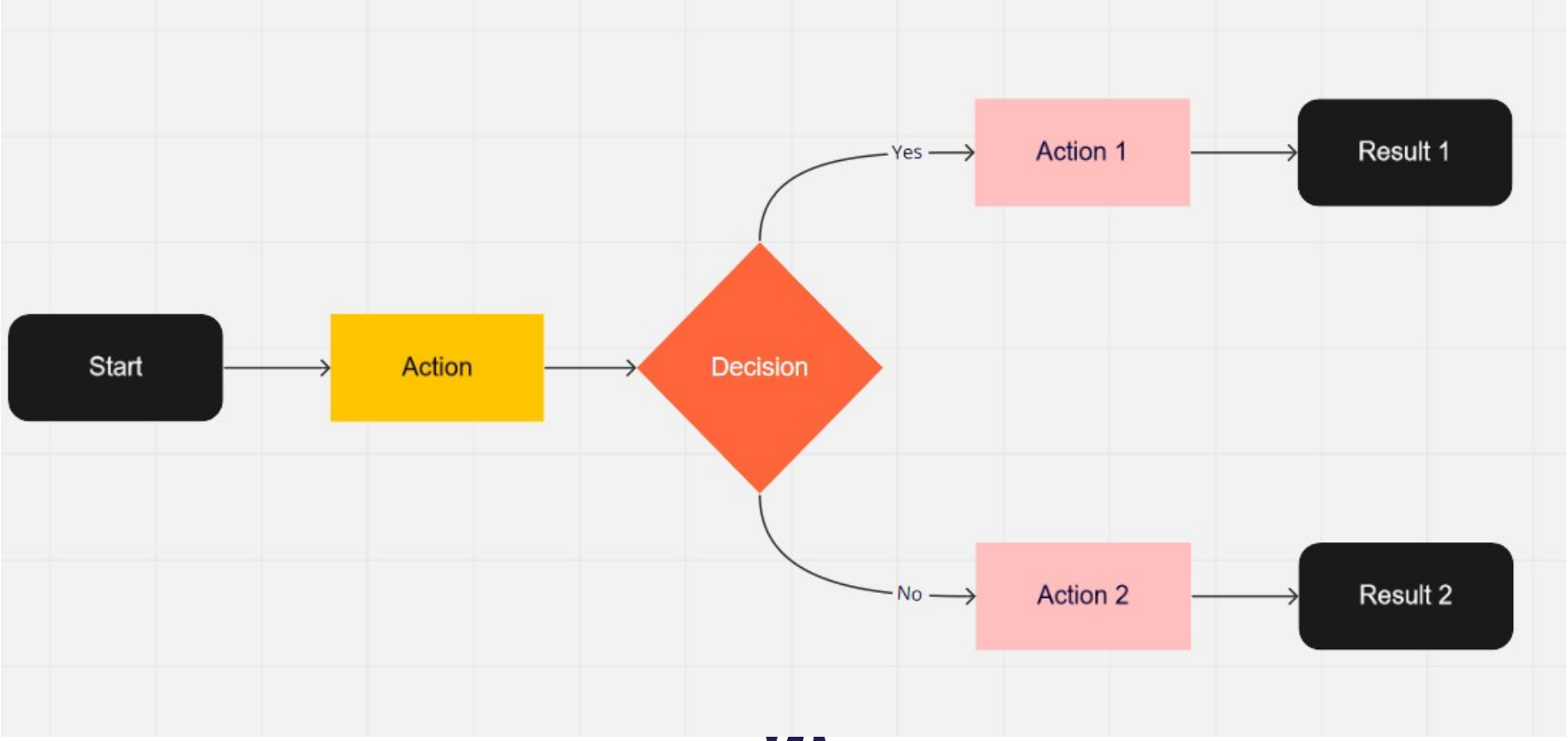
AALBORG UNIVERSITY
DENMARK

Algorithms

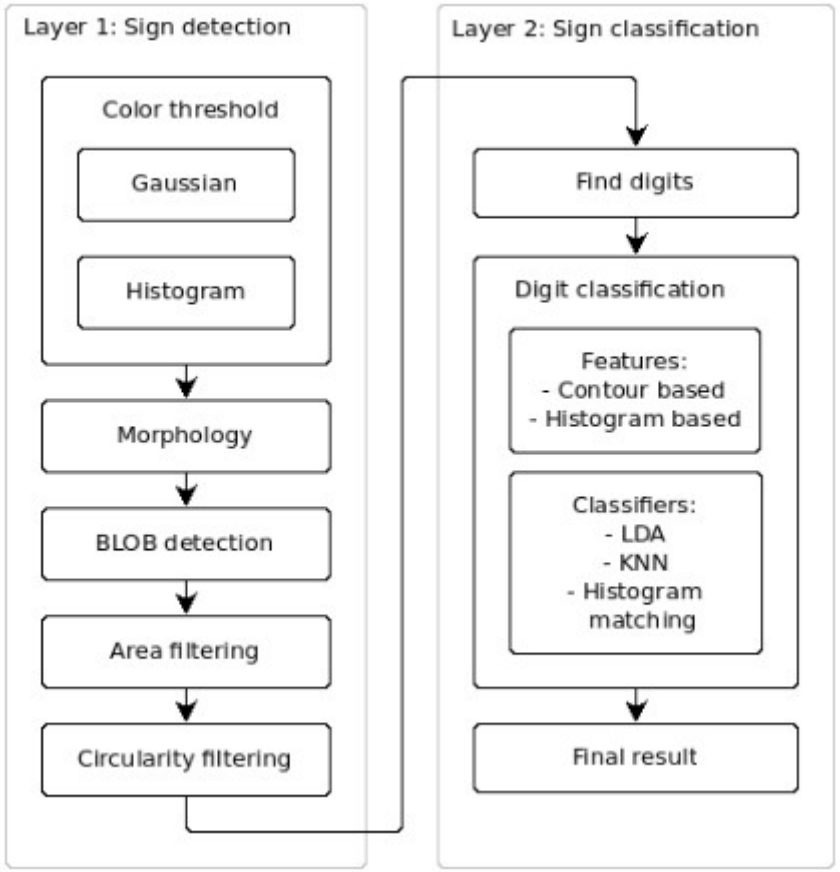
- Addition of numbers

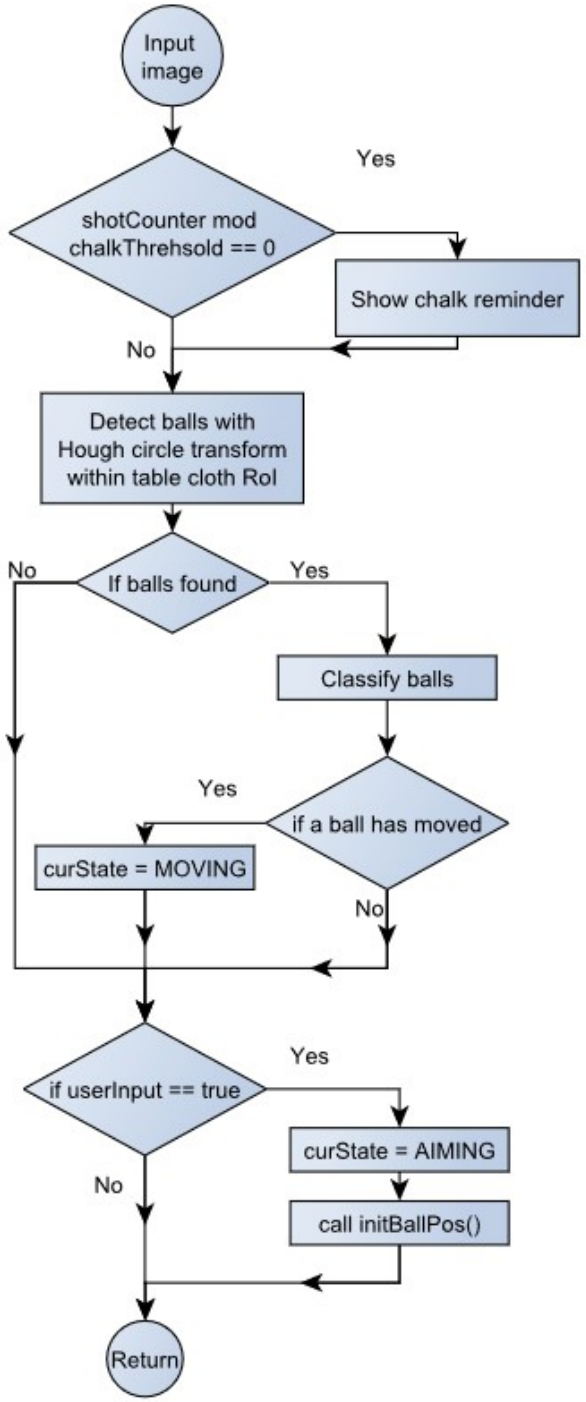
$$\begin{array}{r} {}^{1}58126 \\ +\,15473 \\ \hline 73599 \\ \hline\hline \end{array}$$

Flowcharts and block diagrams



Flowcharts and block diagrams





AALBORG UNIVERSITY
DENMARK

Outline your plan with comments

```
9   # Draw the ground
10  # Draw the bottom of the house
11  # Draw two walls
12  # Draw the roof
```

Programming languages

- What did most of the algorithms, flow charts and comments have in common?
  - They had a sequence of steps to achieve a goal
  - They were intended to be read by a **HUMAN**
  - Now we move to languages intended for a **COMPUTER**



Programming languages

- What is a programming language?

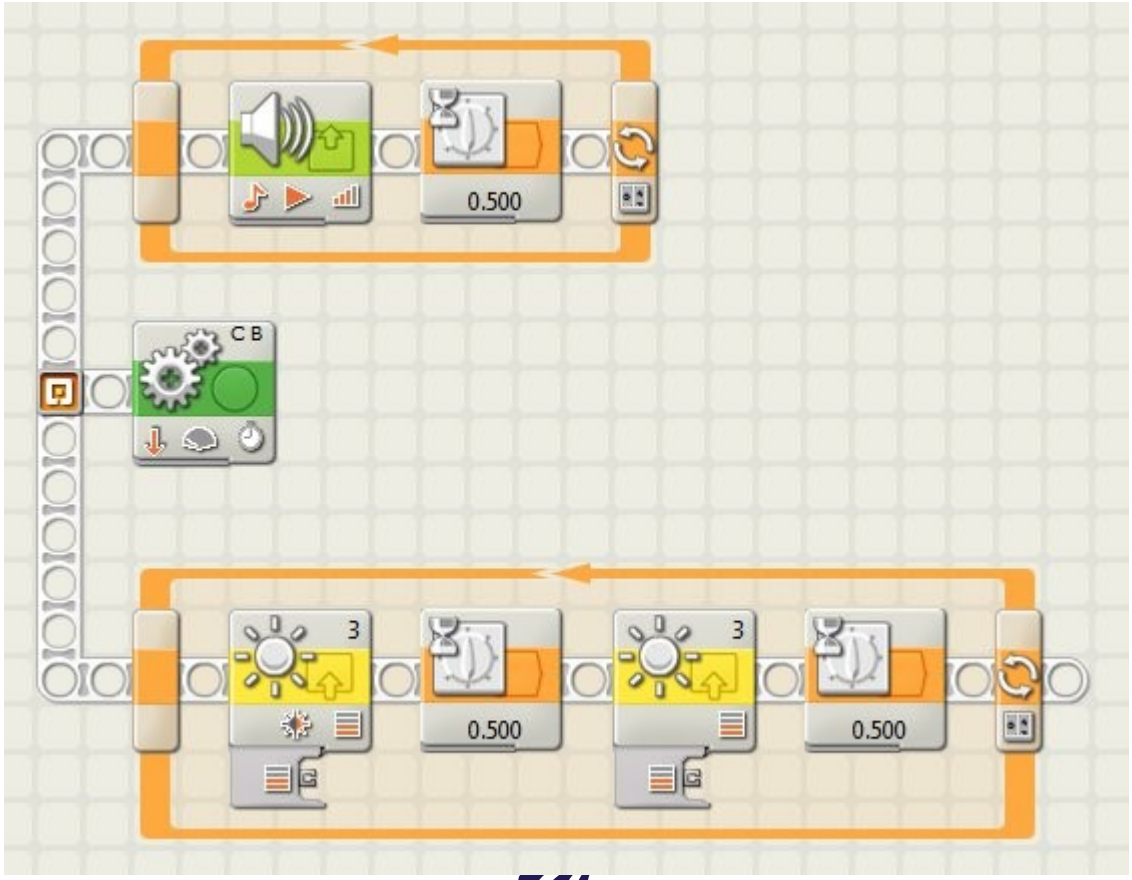
A language for specifying algorithms to a computer

- Programming languages belong to different paradigms and levels of abstraction
- Paradigms
  - Textual languages and graphical languages
  - Imperative, functional, object-oriented, generic, etc.
- Abstraction level
  - Low abstraction level = closer to the basic instructions known by the computer
  - High abstraction level = closer to human languages



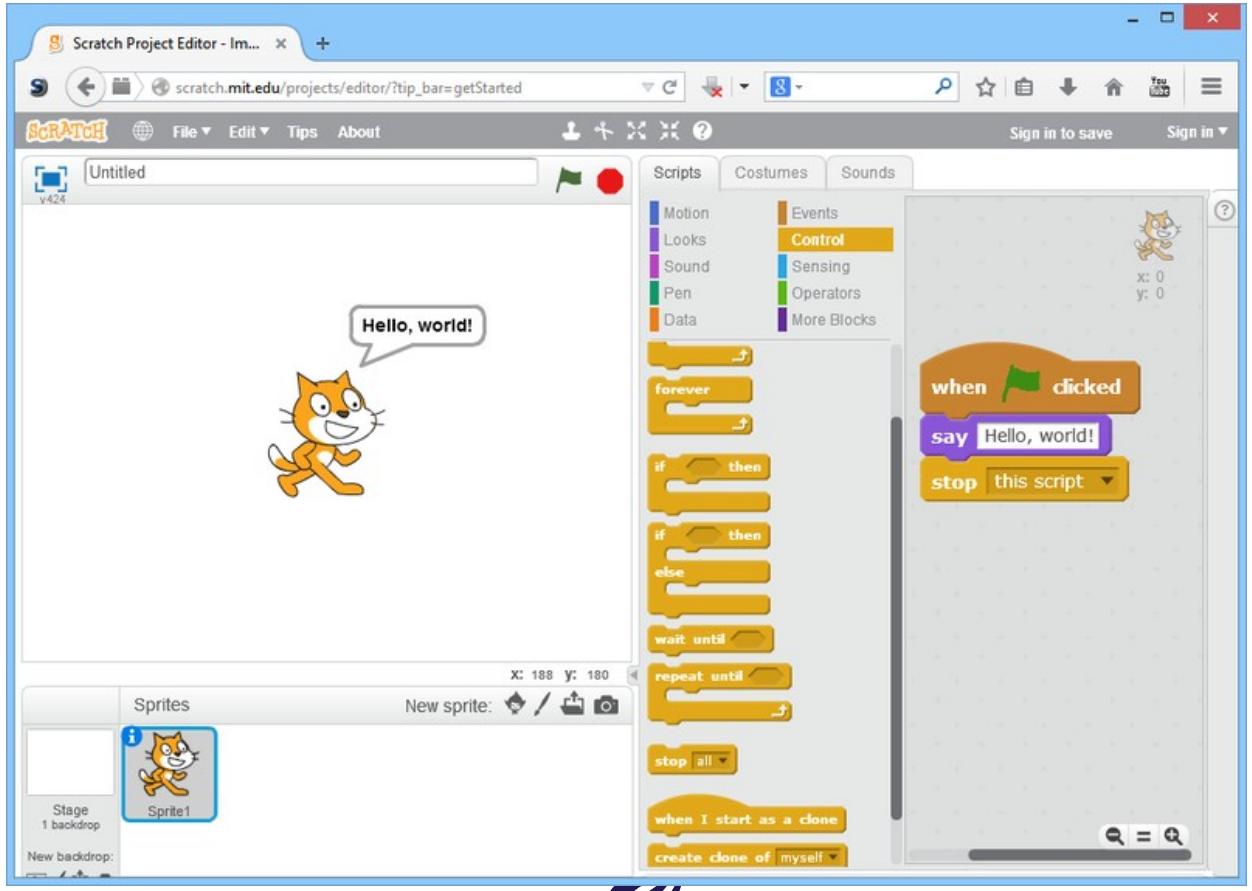
Graphical languages

- Lego Mindstorms NXT-G



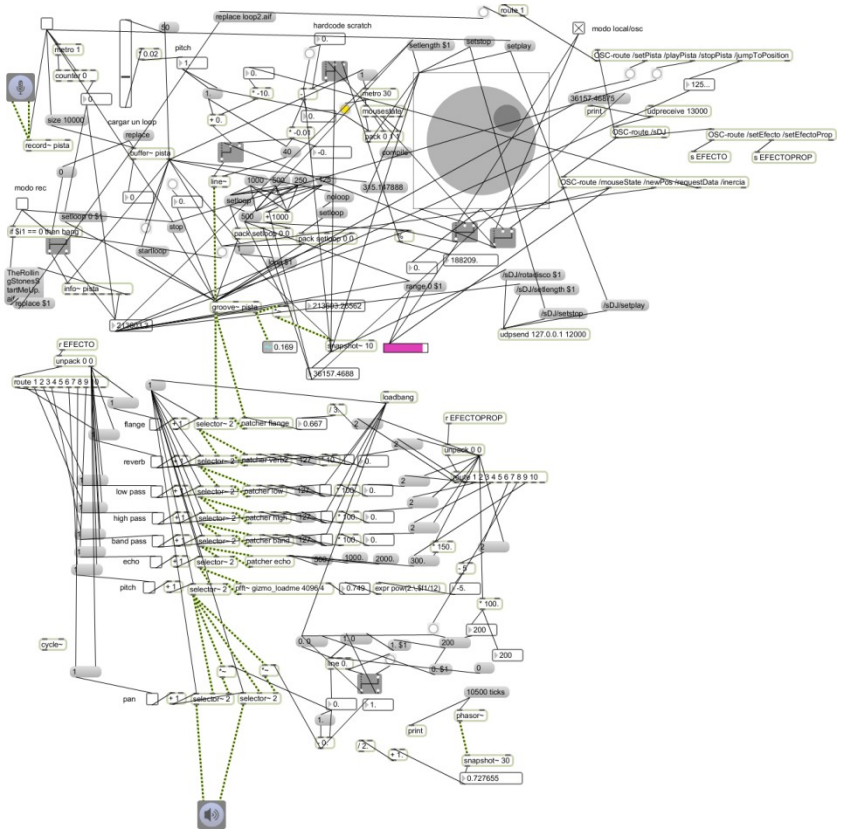
Graphical languages

- Scratch



Graphical languages

- Max/MSP



http://tratadodeintegracion.cc/stream/wp-content/uploads/2009/08/pruena9.png

Textual languages

- Assembly
  - Replaces instruction numbers with words

```
 1      ILOAD j    // i = j + k      0x15 0x02
 2      ILOAD k                      0x15 0x03
 3      IADD                         0x60
 4      ISTORE i                     0x36 0x01
 5      ILOAD i    // if (i < 3)     0x15 0x01
 6      BIPUSH 3                     0x10 0x03
 7      IF_ICMPEQ L1                 0x9F 0x00 0x0D
 8      ILOAD j    // j = j – 1      0x15 0x02
 9      BIPUSH 1                     0x10 0x01
10      ISUB                         0x64
11      ISTORE j                     0x36 0x02
12      GOTO L2                      0xA7 0x00 0x07
13  L1:             BIPUSH 0         // k = 0 0x10 0x00
14      ISTORE k                     0x36 0x03
15  L2:
```

http://withfriendship.com/images/b/9109/assembly-language.jpg



Textual languages

- C
  - Does not look like CPU instructions anymore
  - Requires a *compiler* to be translated into machine code
  - **Hugely** influential language by Dennis Ritchie from 1972

```c
/* Park-Miller "minimal standard" 31 bit
 * pseudo-random number generator, implemented
 * with David G. Carta's optimisation: with
 * 32 bit math and without division.
 */

long unsigned int rand31_next()
{
    long unsigned int hi, lo;

    lo = 16807 * (seed & 0xFFFF);
    hi = 16807 * (seed >> 16);

    lo += (hi & 0x7FFF) << 16;
    lo += hi >> 15;

    if (lo > 0x7FFFFFFF) lo -= 0x7FFFFFFF;

    return ( seed = (long)lo );
}
```

From http://www.firstpr.com.au/dsp/rand31/rand31-park-miller-carta-souce-no-comments.png



Textual languages

- C++
  - An object-oriented and generic extension of C
  - Also a compiled language

```
class classCar {
    protected:
    enumCarMake carMake;
    structTire carTires[4];
    classEngine carMotor;
    classPart carPartsList[100];
    int Speed;
    int Mass;
    public:
    classCar();
    virtual ~classCar();
    int GetSpeed();
    int GetMass ();
};
```

From http://pic.dhe.ibm.com/infocenter/rsarthlp/v7r5m1/topic/com.ibm.xtools.viz.cpp.doc/images/codeasso.gif



Textual languages

- Java
  - Java is compiled to Java byte code
  - Java byte code is run in the Java Virtual Machine (JVM)

```
public class Graph extends Applet {
    GraphPanel panel;

    public void init() {
    setLayout(new BorderLayout());

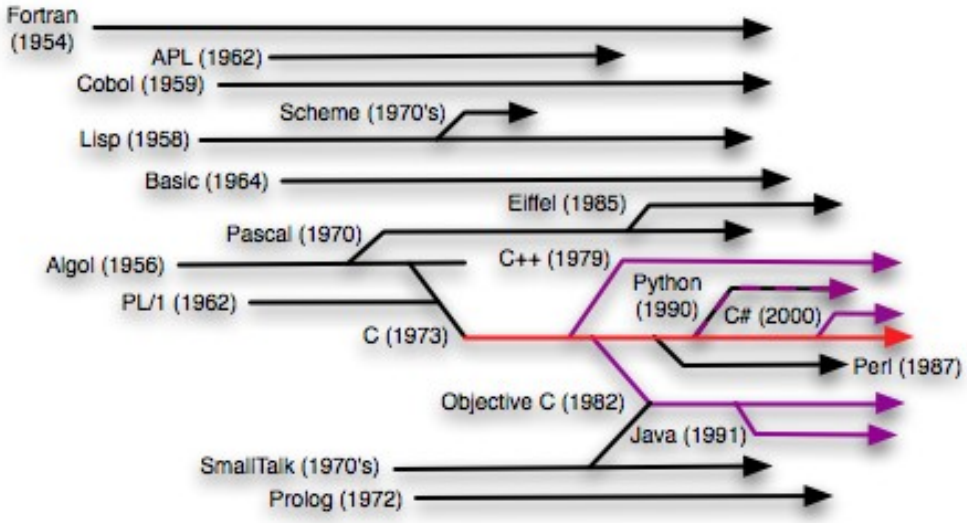
    panel = new GraphPanel(this);
    add("Center", panel);
    Panel p = new Panel();
    add("South", p);
    p.add(new Button("Scramble"));
    p.add(new Button("Shake"));
    p.add(new Checkbox("Stress"));
    p.add(new Checkbox("Random"));

    String edges = getParameter("edges");
    for (StringTokenizer t = new StringTokenizer(edges, ",") ; t.hasMoreTokens() ; ) {
        String str = t.nextToken();
        int i = str.indexOf('-');
        if (i > 0) {
        int len = 50;
        int j = str.indexOf('/');
        if (j > 0) {
            len = Integer.valueOf(str.substring(j+1)).intValue();
            str = str.substring(0, j);
        }
        panel.addEdge(str.substring(0,i), str.substring(i+1), len);
        }
    }
    Dimension d = size();
    String center = getParameter("center");
    if (center != null){
        Node n = panel.nodes[panel.findNode(center)];
        n.x = d.width / 2;
        n.y = d.height / 2;
        n.fixed = true;
    }
    }
```

From http://www.mactech.com/articles/mactech/Vol.12/12.07/Symantec8/img005.gif

Other languages

- Are there more textual languages?
  - Yes, many!
    - (Hundreds, if not thousands)
  - We have only looked at a few of the more common, imperative languages



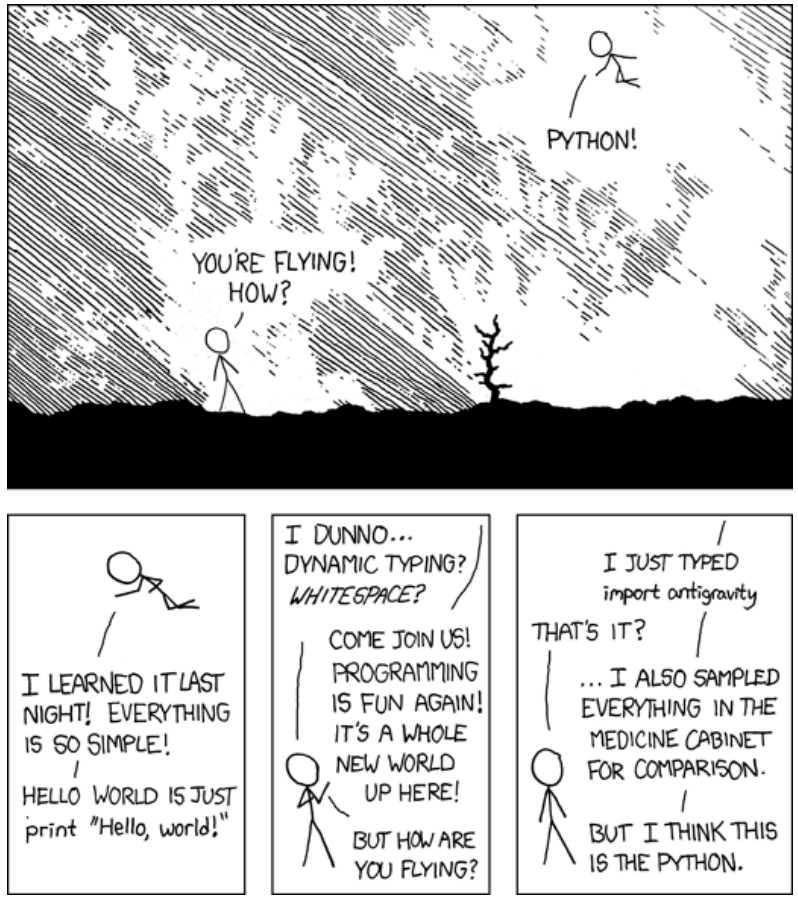
Languages

- So, which language is best?

Depends on the purpose :-)
- that's why so many languages exist!



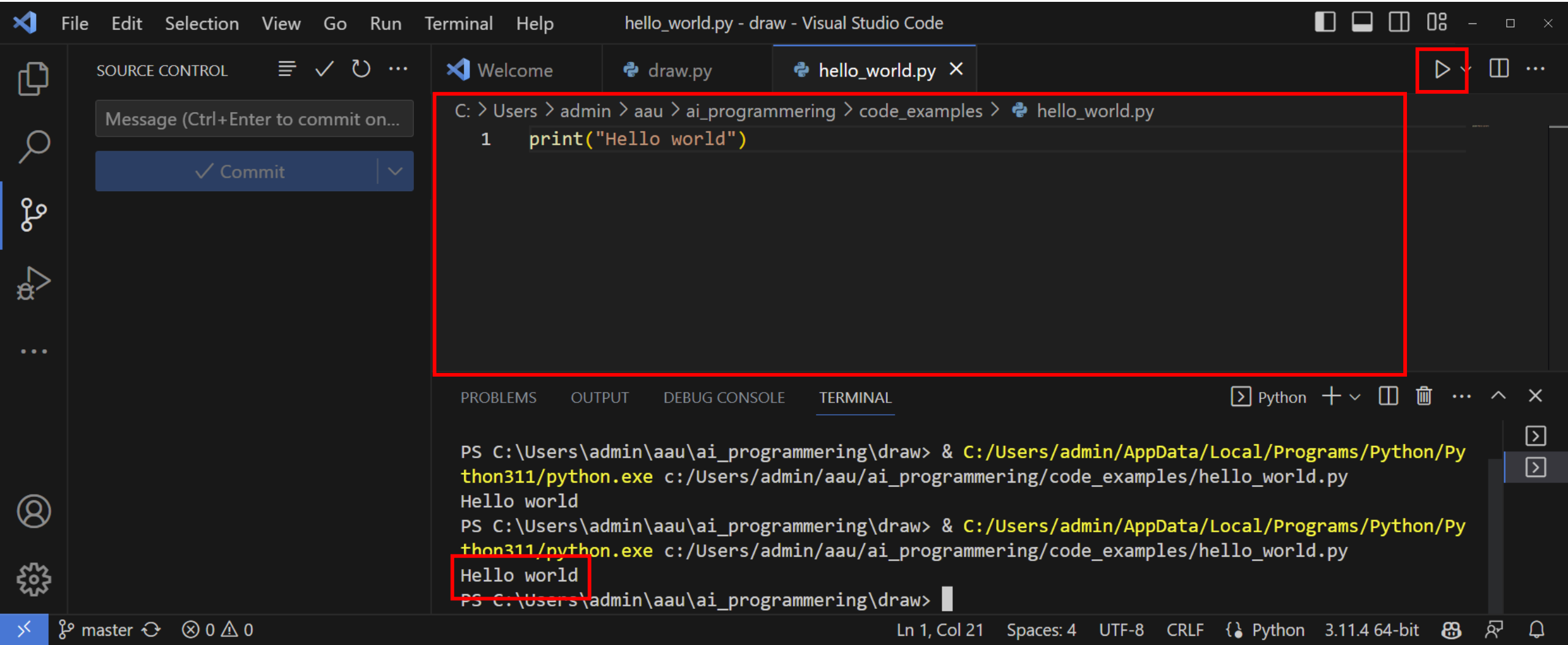
Our language of choice: Python



- High level and easy to get started
- Very powerful – enourmous ecosystem:
  - Data analysis and visualization: Numpy, Scipy, matplotlib, etc.
  - Game development: Pygame
  - Web development: Django
  - There is basically a package for anything you would ever want to do.
- Very well documented

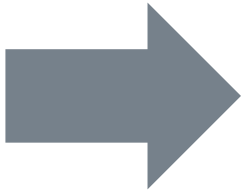
- (Disadvantage: Can be slow)

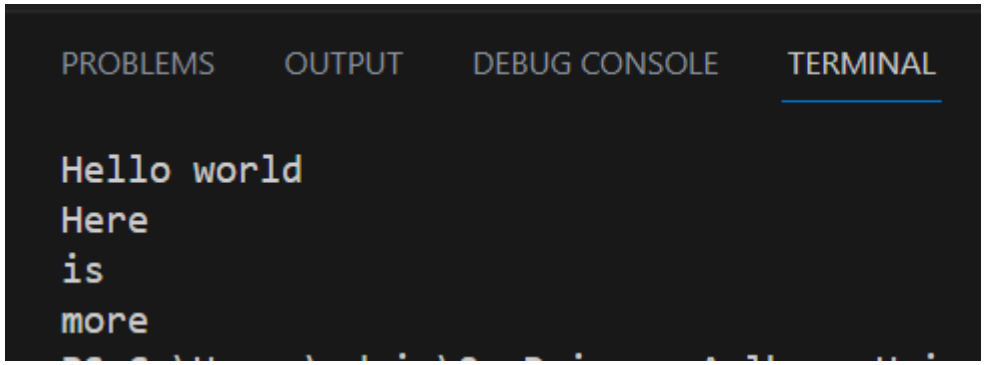
Programming in practice



Execution order

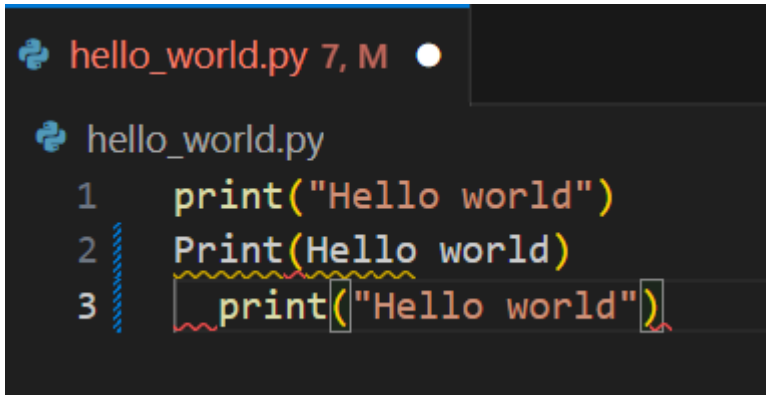






Mind the details!

- CaSE sEnsITiVity
- Paranthesis in the right places, quotes in the right places
- Python is white-space sensitive



Diving into the code

Function name Function parameter(s)

```
print("Hello world")
```

Function call

Method name Method parameter(s)

```
"Hello world".lower()
```

Object Method call

Diving into the code

```
print(“Hello world”.lower())
```



Functions

```
print(“Hello world”)

move(7, 2)

QuitProgram()
```



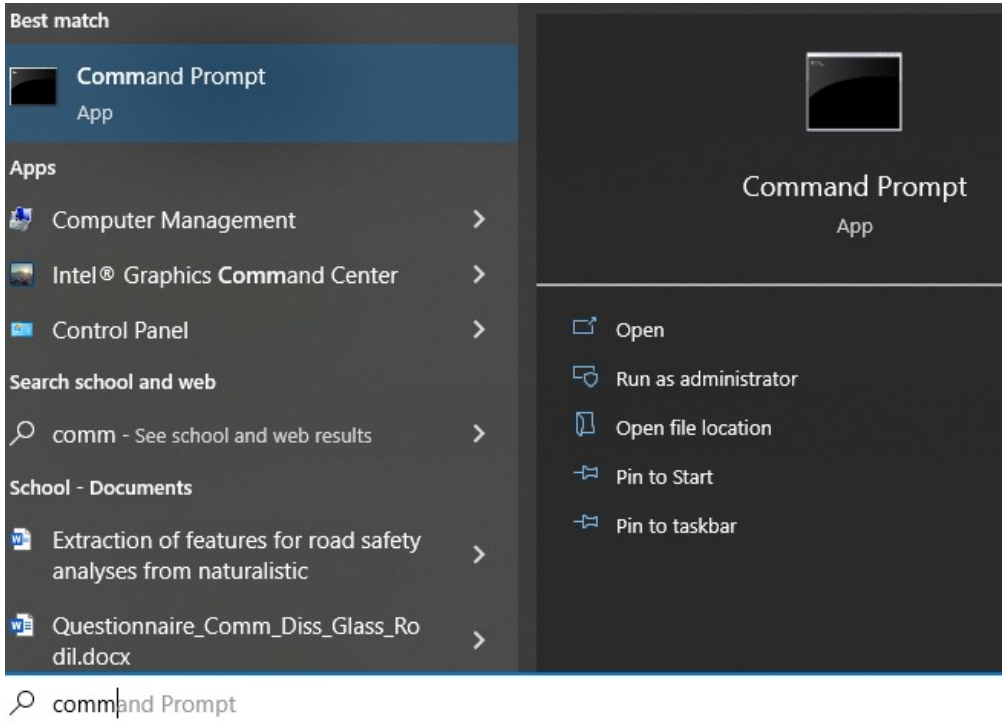
Where do functions come from?

- Standard library
- External libraries
- Make your own!



Installing external libraries

- `pip install` [library name] → e.g. `pip install pygame`
- Import → e.g. `import pygame`




```
admin@DESKTOP-MOA1RPJ C:\Users\admin\OneDrive - Aalborg Universitet
$ pip install pygame
Collecting pygame
  Downloading pygame-2.5.1-cp311-cp311-win_amd64.whl (10.6 MB)
     ---------------------------------------- 10.6/10.6 MB 3.6 MB/s eta 0:00:00
Installing collected packages: pygame
Successfully installed pygame-2.5.1

[notice] A new release of pip is available: 23.1.2 -> 23.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

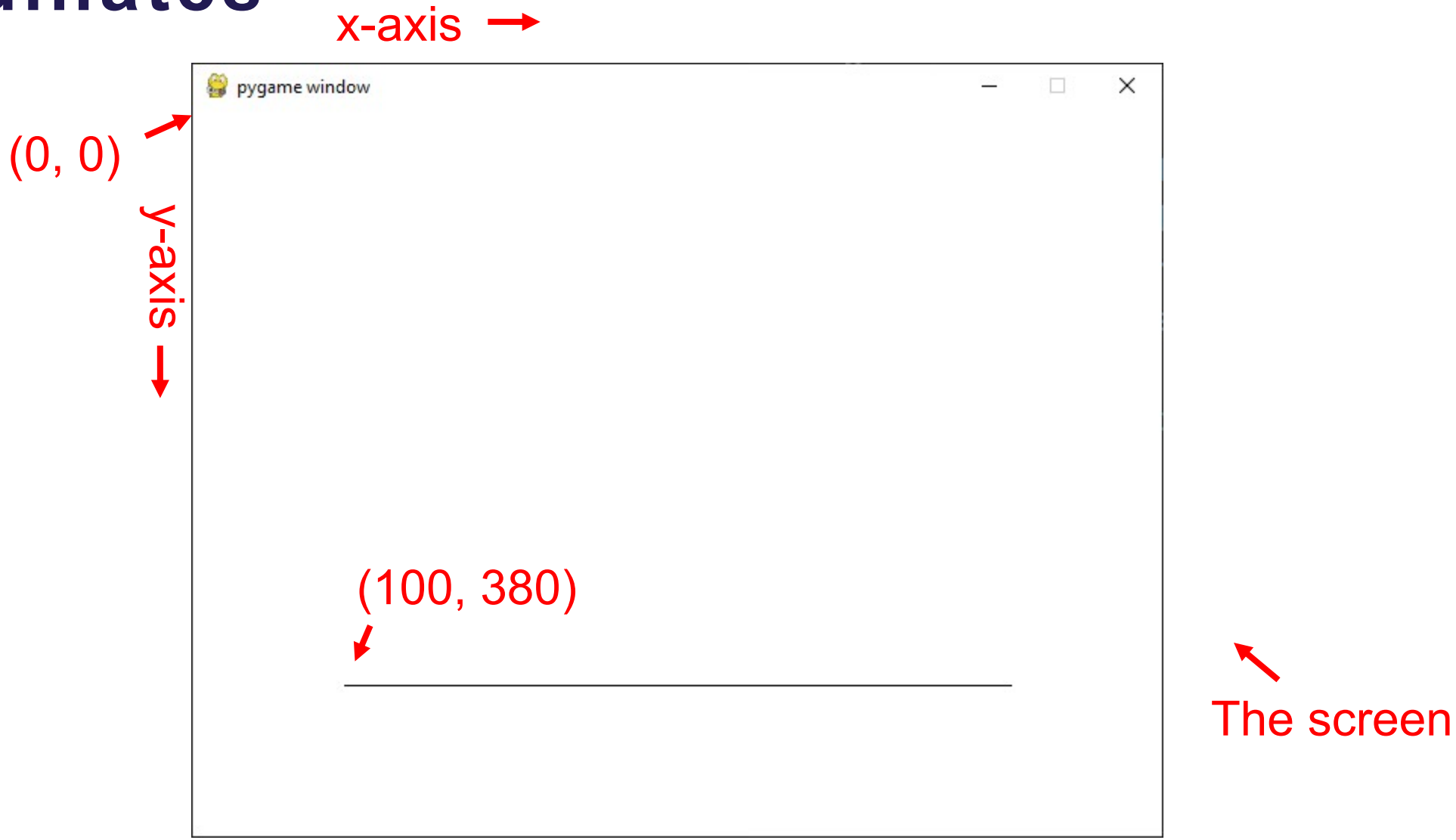
admin@DESKTOP-MOA1RPJ C:\Users\admin\OneDrive - Aalborg Universitet
$
```

A quick detour: Comments

```
# This is a comment. Anything goes here.

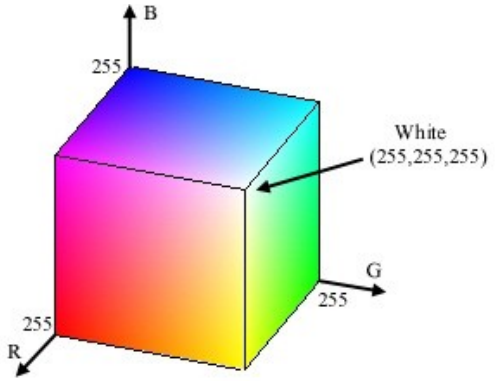
''' This is a comment.
On
Multiple
Lines
'''
```

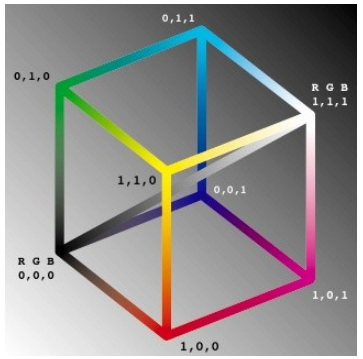
A quick detour: Coordinates

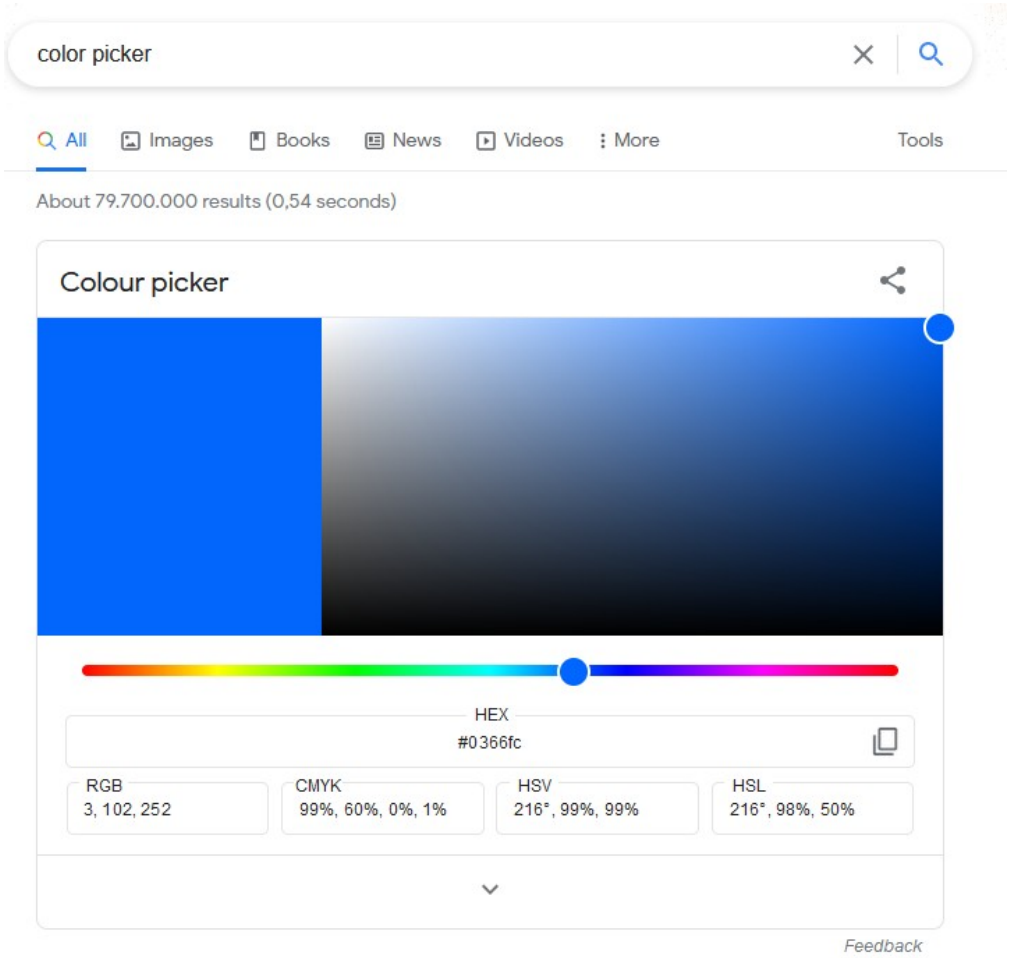


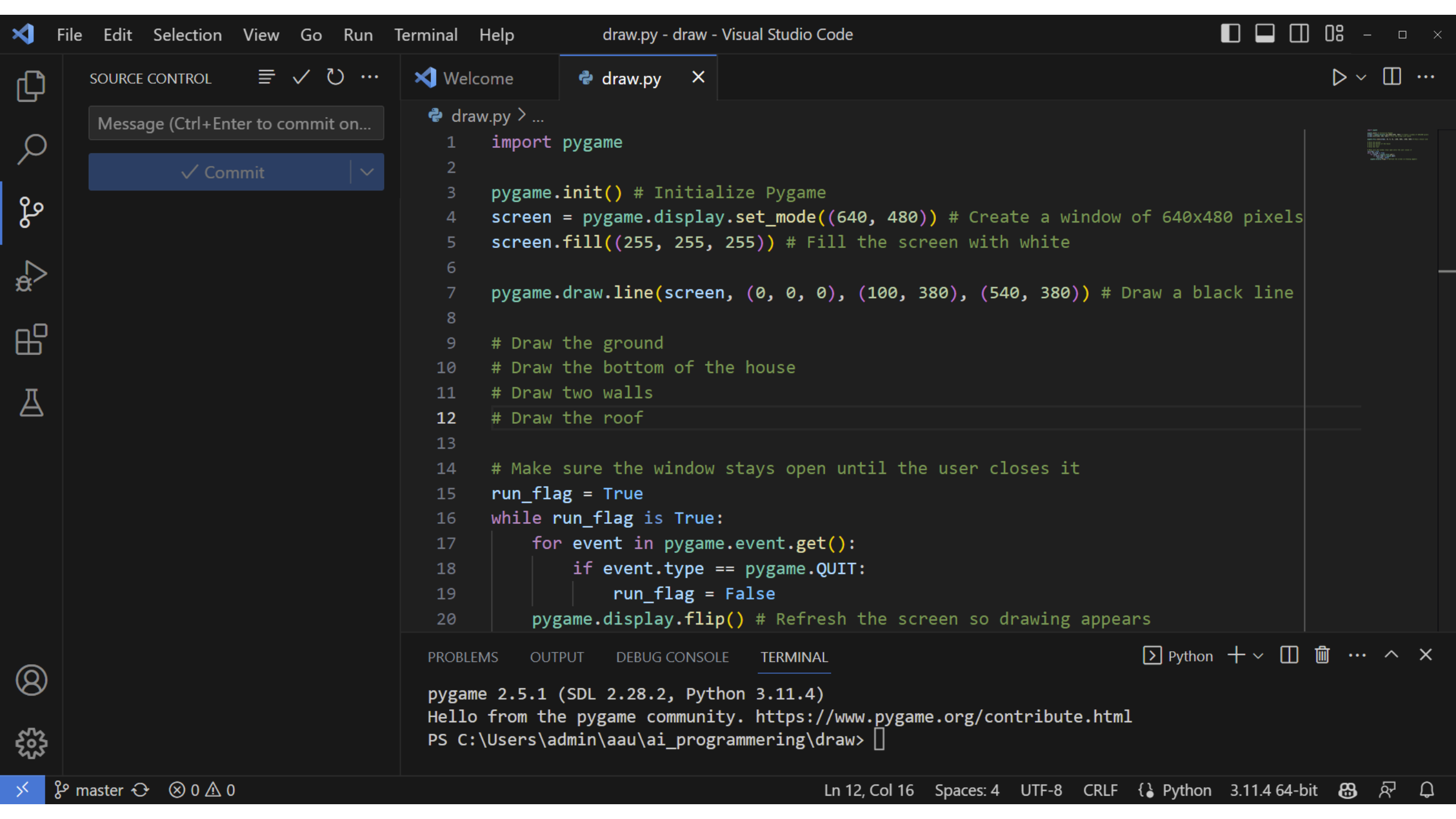
A quick detour: Colors

- RGB – 3 or 4 values: Red, green, blue, and optionally alpha





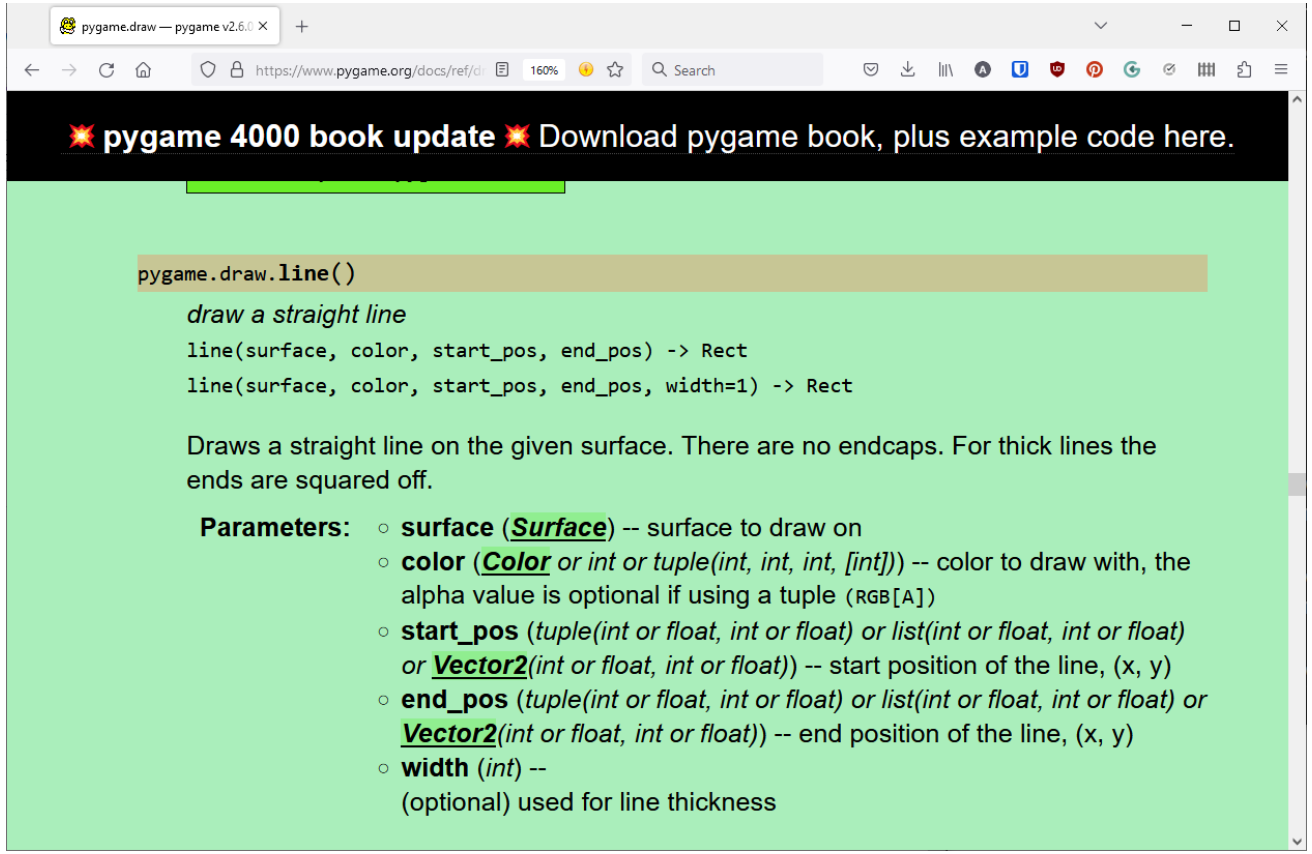



File Edit Selection View Go Run Terminal Help
draw.py - draw - Visual Studio Code
SOURCE CONTROL
Message (Ctrl+Enter to commit on...
Commit
Welcome
draw.py
draw.py > ...
1 import pygame
2
3 pygame.init() # Initialize Pygame
4 screen = pygame.display.set_mode((640, 480)) # Create a window of 640x480 pixels
5 screen.fill((255, 255, 255)) # Fill the screen with white
6
7 pygame.draw.line(screen, (0, 0, 0), (100, 380), (540, 380)) # Draw a black line
8
9 # Draw the ground
10 # Draw the bottom of the house
11 # Draw two walls
12 # Draw the roof
13
14 # Make sure the window stays open until the user closes it
15 run_flag = True
16 while run_flag is True:
17 for event in pygame.event.get():
18 if event.type == pygame.QUIT:
19 run_flag = False
20 pygame.display.flip() # Refresh the screen so drawing appears
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
Python
pygame 2.5.1 (SDL 2.28.2, Python 3.11.4)
Hello from the pygame community. https://www.pygame.org/contribute.html
PS C:\Users\admin\aau\ai_programmering\draw>
master
0 0
Ln 12, Col 16 Spaces: 4 UTF-8 CRLF Python 3.11.4 64-bit

Documentation

- https://docs.python.org
- https://www.pygame.org/docs/

- Just use Google
- Stack Overflow is great
- Ask ChatGPT



Sharing code

- Use Git
- And Github
- … these are not the same thing!!!
  - But they work well together.

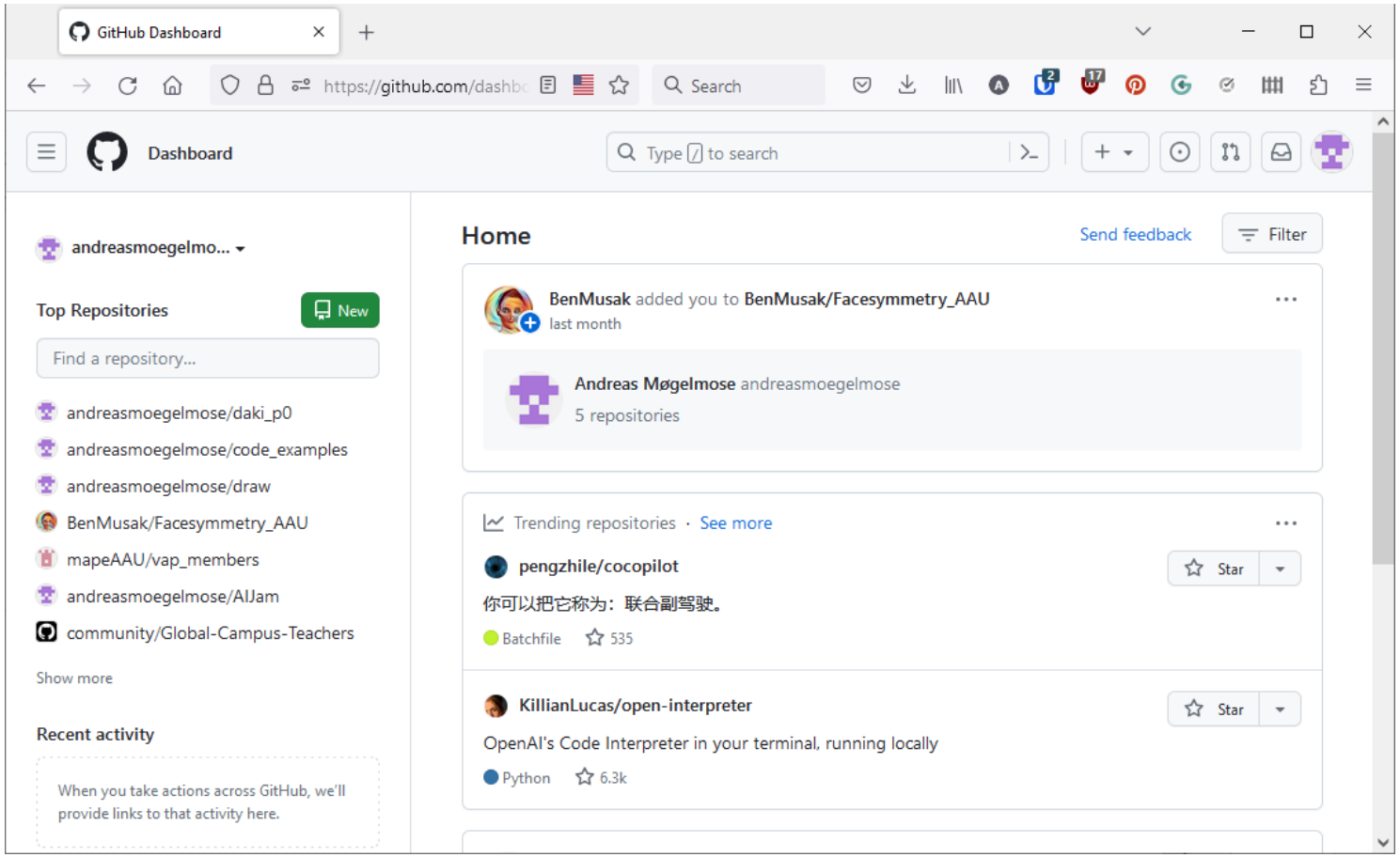
- You use *the program* Git to talk to *the website* GitHub. These are made by different entities.





Github

- Used for sharing code
- Alternatives:
  - Bitbucket
  - Gitlab
  - Your own local server
- Github is where you store your *repository* (*repo* for short) for you and others to access.
- Repos can be public or private



Git

- A program to keep track of any changes you and others make to your code.
- Allows for very fine grained control of changes.
- Handles editing conflicts.

- Use via
  - Terminal/console/command prompt
  - Sublime Merge
  - VS Code

- (Alternatives include SVN and Mercurial)

```
carla@studio:~/git-projects/cpputest$ git status
On branch newbranch
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        deleted:    CppUTest.mak
        modified:   scripts/UnityTemplates/ClassNameCTest.cpp
        modified:   scripts/appveyor_ci_build.ps1

no changes added to commit (use "git add" and/or "git commit -a")
carla@studio:~/git-projects/cpputest$ git add *
carla@studio:~/git-projects/cpputest$ git status
On branch newbranch
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

        modified:   scripts/UnityTemplates/ClassNameCTest.cpp
        modified:   scripts/appveyor_ci_build.ps1

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

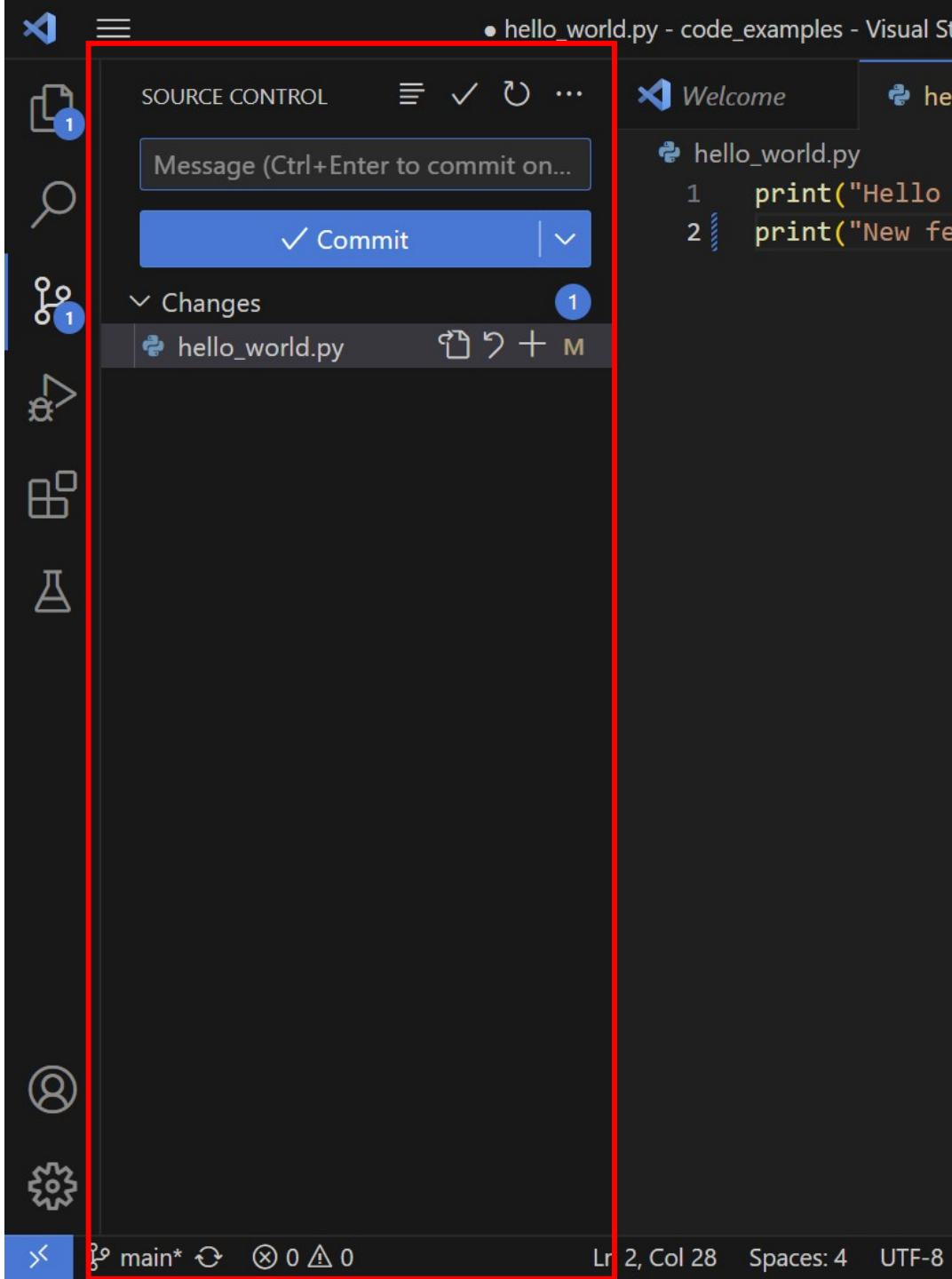
        deleted:    CppUTest.mak

carla@studio:~/git-projects/cpputest$
```



Git workflow

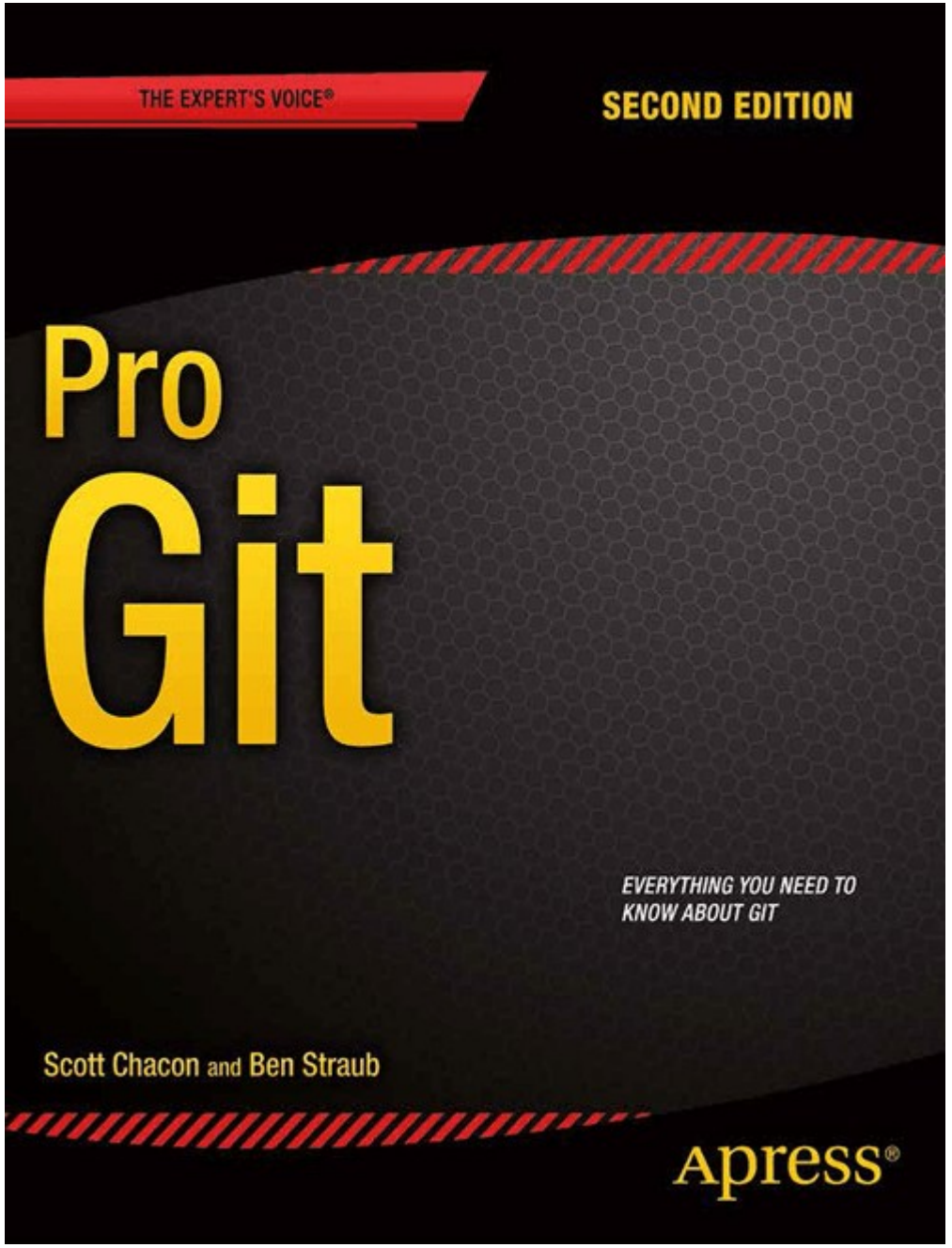
- Make changes in code
- *Add* all related changes
  - This tells Git you want to save them.
- *Commit* your changes along with a message describing them
  - Now your local Git knows about the changes.
- *Push* your changes to a server (probably Github)
  - Now the world knows about your changes
- *Pull* any changes others have made to your shared code.



AALBORG UNIVERSITY
DENMARK

Git advantages

- Anything and everything can be undone
- Allows multiple people to work on the same software simultaneously without disturbing each other (through *branching*)
- Can be used to find bugs (errors in code)
- Can outline a full history of your code
- … and many other things





Exercise time!